SQL for Interviews

Complete Interview Training Guide

Level 1 (Basics) through Level 11 (Classic Interview Problems)

Simple English answers with step-by-step SQL techniques for every topic

Prepared for interview-ready, job-ready SQL mastery

PRACTICE TABLES USED IN THIS GUIDE

Most examples use these simple tables. Create them in any SQL tool (MySQL, PostgreSQL, SQL Server) and try each query yourself.

Table	Columns
employees	emp_id, name, dept, city, salary, manager_id, hire_date
customers	customer_id, name, email, phone, city
orders	order_id, customer_id, order_date, amount, status
logins	user_id, login_date
monthly_sales / daily_sales	month or sale_date, revenue or amount

DATABASE NOTE

SQL is almost the same everywhere, but some commands change by database. Examples: LIMIT (MySQL, PostgreSQL) vs TOP (SQL Server), DATE_TRUNC (PostgreSQL) vs DATE_FORMAT (MySQL), LEN (SQL Server) vs LENGTH (others). Each topic mentions the difference where it matters. Learn one database well, and know that the others have small differences.

HOW EACH TOPIC IS WRITTEN

Every topic has the same parts: Definition, Simple Explanation, Example (with SQL code), Technique (Step-by-Step), Interview Answer, Follow-up Questions, Common Mistake and Related topics. Every level ends with a Quick Revision table, Top Interview Questions and Practical Scenarios.

LEVEL 1 - SQL BASICS

SELECT

Definition: SELECT is used to read (fetch) data from one or more columns of a table.

Simple Explanation: It is the most used SQL command. You tell SQL which columns you want and from which table. Use * to get all columns, but it is better to write the column names you need.

Example: Show the name and salary of all employees.

```
SELECT name, salary
FROM employees;
```

Technique (Step-by-Step): 1. Write SELECT and list the columns you need, separated by commas. 2. Write FROM and then the table name. 3. End the query with a semicolon. 4. Use AS to give a column a clear name, like salary AS monthly_pay.

Interview Answer: “SELECT is the command I use to read data from a table. I always pick only the columns I need instead of using SELECT *, because it is faster and easier to read.”

Follow-up Qs:
- **Q:** Why avoid SELECT *? **A:** It reads every column, so it is slow on big tables. It can also break reports if someone adds a new column.
- **Q:** What is an alias? **A:** An alias is a temporary name for a column or table, written with AS. It makes results easier to read.

Common Mistake: Using SELECT * in real reports and dashboards.

Related: Leads to DISTINCT, WHERE and ORDER BY.

DISTINCT

Definition: DISTINCT removes duplicate rows from the result.

Simple Explanation: If a column has repeated values, DISTINCT shows each value only once. Use it to see the unique values, like the list of cities your customers live in.

Example: Show every city only once from the customers table.

```
SELECT DISTINCT city
FROM customers;
```

Technique (Step-by-Step): 1. Write SELECT DISTINCT. 2. List the column or columns. 3. When you list many columns, DISTINCT checks the whole combination, not one column.

Interview Answer: “DISTINCT gives me only the unique values from a column. I use it to quickly check categories, like all the different cities or product types in a table.”

Follow-up Qs:
- **Q:** Does DISTINCT work on many columns? **A:** Yes. It removes rows where the full combination of columns is repeated.
- **Q:** How do you count unique values? **A:** Use COUNT(DISTINCT column).

Common Mistake: Thinking DISTINCT works on only the first column when many columns are listed.

Related: Related to GROUP BY, which can also give unique values.

WHERE

Definition: WHERE filters rows. Only rows that meet the condition are returned.

Simple Explanation: Without WHERE, SQL returns every row. With WHERE, you keep only the rows you need. You can join many conditions with AND, OR and NOT.

Example: Show employees from Delhi who earn more than 50000.

```
SELECT name, salary
FROM employees
WHERE city = 'Delhi'
  AND salary > 50000;
```

Technique (Step-by-Step): 1. Write your SELECT and FROM first. 2. Add WHERE and write one condition. 3. Use AND when all conditions must be true. 4. Use OR when any one condition is enough. 5. Use brackets when you mix AND and OR.

Interview Answer: “WHERE filters the rows before anything else is calculated. I use it to keep only the records I need, like orders from one city or one month.”

Follow-up Qs:
- **Q:** What is the order of AND and OR? **A:** AND runs before OR. Use brackets to be safe.
- **Q:** Can you use aggregate functions in WHERE? **A:** No. Use HAVING for that.

Common Mistake: Forgetting brackets when mixing AND with OR, which gives wrong rows.

Related: Leads to HAVING (Level 2).

ORDER BY

Definition: ORDER BY sorts the result in ascending (ASC) or descending (DESC) order.

Simple Explanation: By default, SQL does not guarantee any order. ORDER BY makes the order fixed. ASC is the default. You can sort by many columns.

Example: Show employees from highest salary to lowest. If salary is same, sort by name.

```
SELECT name, salary
FROM employees
ORDER BY salary DESC, name ASC;
```

Technique (Step-by-Step): 1. Write ORDER BY at the end of the query. 2. Add the first column and ASC or DESC. 3. Add a comma and a second column to break ties. 4. You can also use the column number, like ORDER BY 2.

Interview Answer: “ORDER BY sorts my result. I use DESC to find the top values, like top-selling products, and I can add a second column to break ties.”

Follow-up Qs:
- **Q:** Is ORDER BY the default order? **A:** No. Without ORDER BY the order is not guaranteed.
- **Q:** Can ORDER BY use an alias? **A:** Yes. ORDER BY runs after SELECT, so it can use aliases.

Common Mistake: Assuming data comes back sorted without using ORDER BY.

Related: Pairs with LIMIT/TOP.

LIMIT / TOP

Definition: LIMIT (or TOP) returns only the first few rows of the result.

Simple Explanation: Use it to see a small sample or the top N rows. Different databases use different words: LIMIT in MySQL and PostgreSQL, TOP in SQL Server, FETCH FIRST in Oracle.

Example: Show the 5 highest paid employees.

```
-- MySQL / PostgreSQL
SELECT name, salary FROM employees
ORDER BY salary DESC
LIMIT 5;

-- SQL Server
SELECT TOP 5 name, salary FROM employees
ORDER BY salary DESC;
```

Technique (Step-by-Step): 1. First sort the data with ORDER BY. 2. Add LIMIT 5 at the end (or TOP 5 right after SELECT in SQL Server). 3. For paging, use LIMIT 5 OFFSET 10 to skip the first 10 rows.

Interview Answer: “LIMIT or TOP gives me only the first N rows. I use it together with ORDER BY to get top-N results, like the top 5 customers by sales.”

Follow-up Qs:
- **Q:** Why use ORDER BY with LIMIT? **A:** Without ORDER BY, the first N rows are random.
- **Q:** What is OFFSET? **A:** OFFSET skips a number of rows. It is used for paging.

Common Mistake: Using LIMIT without ORDER BY and getting random rows.

Related: Used in Nth highest salary problems (Level 11).

LIKE

Definition: LIKE searches for a text pattern using wildcards.

Simple Explanation: The % sign means any number of characters. The _ sign means exactly one character. Use LIKE when you do not know the full text.

Example: Find customers whose name starts with A.

```
SELECT name FROM customers
WHERE name LIKE 'A%';

-- names with 'son' anywhere
WHERE name LIKE '%son%';
```

Technique (Step-by-Step): 1. Write WHERE column LIKE. 2. Put the pattern in single quotes. 3. 'A%' means starts with A. '%A' means ends with A. '%A%' means contains A. 4. Use NOT LIKE to exclude a pattern.

Interview Answer: “LIKE finds text that matches a pattern. I use % for any characters and _ for one character, for example to find all emails ending with gmail.com.”

Follow-up Qs:
- **Q:** Difference between % and _ ? **A:** % matches many characters. _ matches exactly one.
- **Q:** Is LIKE case-sensitive? **A:** It depends on the database. In PostgreSQL it is case-sensitive; use ILIKE for case-free search.

Common Mistake: Using a leading % (like '%abc') on a big table. It cannot use an index and is slow.

Related: Related to string functions (Level 7).

IN

Definition: IN checks if a value matches any value in a list.

Simple Explanation: It is a short way to write many OR conditions. It makes the query cleaner.

Example: Show employees from Delhi, Mumbai or Chennai.

```
SELECT name, city FROM employees
WHERE city IN ('Delhi', 'Mumbai', 'Chennai');
```

Technique (Step-by-Step): 1. Write WHERE column IN. 2. Put the values inside brackets, separated by commas. 3. Use NOT IN to exclude values. 4. You can also put a subquery inside the brackets.

Interview Answer: “IN is a clean way to match a value against a list. It replaces long OR chains, like city = 'Delhi' OR city = 'Mumbai'.”

Follow-up Qs:
- **Q:** IN vs many ORs? **A:** They give the same result. IN is shorter and easier to read.
- **Q:** What is the danger of NOT IN? **A:** If the list has a NULL, NOT IN returns no rows at all.

Common Mistake: Using NOT IN with a subquery that can return NULL.

Related: Leads to EXISTS vs IN (Level 4).

BETWEEN

Definition: BETWEEN checks if a value falls inside a range. Both ends are included.

Simple Explanation: Use it for number ranges and date ranges. It is the same as writing >= and <= together.

Example: Show orders placed in January 2024.

```
SELECT * FROM orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-01-31';
```

Technique (Step-by-Step): 1. Write WHERE column BETWEEN. 2. Write the low value, then AND, then the high value. 3. Remember that both low and high are included.

Interview Answer: "BETWEEN filters values inside a range, and it includes both ends. I mostly use it for dates and salary ranges."

Follow-up Qs:
- **Q:** Is BETWEEN inclusive? **A:** Yes. Both start and end values are included.
- **Q:** What is the trap with dates that have time? **A:** BETWEEN '2024-01-01' AND '2024-01-31' misses orders on Jan 31 after 00:00. Use < '2024-02-01' instead.

Common Mistake: Forgetting that the end date with time can be missed.

Related: Related to date functions (Level 7).

IS NULL / IS NOT NULL

Definition: IS NULL checks for missing values. NULL means unknown or no value.

Simple Explanation: NULL is not zero and not an empty text. You cannot use = with NULL. You must use IS NULL.

Example: Find employees who have no manager.

```
SELECT name FROM employees
WHERE manager_id IS NULL;
```

Technique (Step-by-Step): 1. Write WHERE column IS NULL to find missing values. 2. Write IS NOT NULL to find filled values. 3. Never write = NULL. It never gives true.

Interview Answer: "NULL means a missing value. I always use IS NULL or IS NOT NULL to check it, because column = NULL never works."

Follow-up Qs:
- **Q:** Why does = NULL not work? **A:** NULL is unknown. Anything compared to unknown is unknown, not true.
- **Q:** Does COUNT(column) count NULLs? **A:** No. COUNT(column) skips NULLs. COUNT(*) counts all rows.

Common Mistake: Writing WHERE column = NULL and getting zero rows.

Related: Leads to COALESCE and NULLIF (Level 6).

Level 1 - Quick Revision

Command	Purpose
SELECT	Pick the columns to read
DISTINCT	Remove duplicate rows
WHERE	Filter rows
ORDER BY	Sort the result
LIMIT / TOP	Return only first N rows
LIKE	Search a text pattern
IN	Match any value in a list
BETWEEN	Match a range (ends included)
IS NULL	Find missing values

Key rule: Never use = NULL. Always use IS NULL.

Top Interview Questions: (1) Difference between WHERE and HAVING. (2) How do you check for NULL values? (3) What is the difference between % and _ in LIKE?

Practical Scenarios: (1) Show the 10 newest orders - ORDER BY + LIMIT. (2) Find customers with no email - IS NULL. (3) List all unique cities - DISTINCT.

LEVEL 2 - AGGREGATES AND GROUPING

COUNT

Definition: COUNT returns the number of rows or values.

Simple Explanation: COUNT(*) counts all rows. COUNT(column) counts only rows where that column is not NULL. COUNT(DISTINCT column) counts unique values.

Example: Count all employees, and count employees who have a manager.

```
SELECT COUNT(*) AS total_rows,
       COUNT(manager_id) AS with_manager,
       COUNT(DISTINCT city) AS unique_cities
FROM employees;
```

Technique (Step-by-Step): 1. Use COUNT(*) when you want the total number of rows. 2. Use COUNT(column) to ignore NULLs. 3. Use COUNT(DISTINCT column) for unique values. 4. Combine with GROUP BY to count per group.

Interview Answer: “COUNT tells me how many rows there are. I use COUNT(*) for total rows and COUNT(column) when I want to skip missing values.”

Follow-up Qs:
- **Q:** COUNT(*) vs COUNT(column)? **A:** COUNT(*) counts every row. COUNT(column) skips NULLs.
- **Q:** What does COUNT return on an empty table? **A:** It returns 0, not NULL.

Common Mistake: Using COUNT(column) when the column has NULLs and expecting the total row count.

Related: Leads to GROUP BY.

SUM

Definition: SUM adds up all the numbers in a column.

Simple Explanation: It ignores NULL values. If all values are NULL, SUM returns NULL, not 0.

Example: Find the total sales amount.

```
SELECT SUM(amount) AS total_sales
FROM orders;
```

Technique (Step-by-Step): 1. Write SUM(column). 2. Add WHERE if you need a total for only some rows. 3. Add GROUP BY to get a total per group. 4. Use COALESCE(SUM(amount), 0) to show 0 instead of NULL.

Interview Answer: “SUM adds numbers in a column and ignores NULLs. I use it with GROUP BY to get totals like sales per region.”

Follow-up Qs:
- **Q:** What if all values are NULL? **A:** SUM returns NULL. Use COALESCE to turn it into 0.
- **Q:** Can SUM work on text? **A:** No. It works only on numbers.

Common Mistake: Not handling the NULL result when no rows match.

Related: Pairs with AVG, MIN, MAX.

AVG

Definition: AVG returns the average (mean) of a column.

Simple Explanation: It ignores NULLs. This can change the answer, because NULL rows are not counted in the division.

Example: Find the average salary per city.

```
SELECT city, AVG(salary) AS avg_salary
FROM employees
GROUP BY city;
```

Technique (Step-by-Step): 1. Write AVG(column). 2. Add GROUP BY for each group. 3. Use ROUND(AVG(column), 2) to get clean numbers. 4. Check for NULLs if the result looks odd.

Interview Answer: “AVG gives the mean value, and it skips NULLs. I use ROUND to keep the result clean, and I remember that outliers can pull the average up or down.”

Follow-up Qs:
- **Q:** Does AVG count NULLs as 0? **A:** No. They are skipped. So the average can be higher than expected.
- **Q:** How to treat NULL as 0 in AVG? **A:** Use AVG(COALESCE(column, 0)).

Common Mistake: Forgetting that NULLs are skipped, which changes the average.

Related: Compare with MEDIAN for outliers.

MIN and MAX

Definition: MIN returns the smallest value. MAX returns the largest value.

Simple Explanation: They work on numbers, text and dates. For dates, MIN is the oldest and MAX is the newest.

Example: Find the first and the latest order date.

```
SELECT MIN(order_date) AS first_order,
       MAX(order_date) AS last_order
FROM orders;
```

Technique (Step-by-Step): 1. Write MIN(column) or MAX(column). 2. Add GROUP BY for per-group results. 3. To get the whole row with the max value, use a subquery or a window function.

Interview Answer: “MIN and MAX return the lowest and highest value. I use them for dates, prices and scores, and with GROUP BY for per-group results.”

Follow-up Qs:
- **Q:** Can MAX give the whole row? **A:** No. It returns only the value. Use ORDER BY with LIMIT 1, or a subquery, to get the row.
- **Q:** Do they work on text? **A:** Yes. They use alphabetical order.

Common Mistake: Selecting other columns next to MAX without GROUP BY.

Related: Leads to subqueries (Level 4).

GROUP BY

Definition: GROUP BY puts rows with the same value into groups, so you can run an aggregate on each group.

Simple Explanation: Think of it as 'for each'. For each city, count employees. Every column in SELECT must be either in GROUP BY or inside an aggregate function.

Example: Count employees in each city.

```
SELECT city, COUNT(*) AS emp_count
FROM employees
GROUP BY city
ORDER BY emp_count DESC;
```

Technique (Step-by-Step): 1. Decide the group (for example city). 2. Write that column in SELECT and in GROUP BY. 3. Add an aggregate like COUNT, SUM or AVG. 4. Use HAVING to filter the groups. 5. Group by two columns to get finer groups.

Interview Answer: “GROUP BY splits the data into groups and lets me calculate one result per group, like total sales per region or employees per department.”

Follow-up Qs:
- **Q:** What if a SELECT column is not in GROUP BY? **A:** Most databases give an error, unless it is inside an aggregate function.
- **Q:** Can you GROUP BY many columns? **A:** Yes. Each unique combination becomes one group.

Common Mistake: Putting a column in SELECT but forgetting it in GROUP BY.

Related: Leads to HAVING.

HAVING

Definition: HAVING filters groups after GROUP BY has been done.

Simple Explanation: WHERE filters rows before grouping. HAVING filters groups after grouping. Only HAVING can use aggregate functions like COUNT and SUM.

Example: Show cities that have more than 10 employees.

```
SELECT city, COUNT(*) AS emp_count
FROM employees
GROUP BY city
HAVING COUNT(*) > 10;
```

Technique (Step-by-Step): 1. Write your GROUP BY query first. 2. Add HAVING after GROUP BY. 3. Write the condition using an aggregate, like COUNT(*) > 10. 4. Use WHERE for normal row filters and HAVING only for group filters.

Interview Answer: “HAVING filters groups after aggregation. For example, I use it to show only the customers who placed more than 5 orders.”

Follow-up Qs:
- **Q:** Can HAVING be used without GROUP BY? **A:** Yes, but it is rare. The whole table becomes one group.
- **Q:** Why not use WHERE for aggregates? **A:** WHERE runs before grouping, so the aggregate does not exist yet.

Common Mistake: Using HAVING for a normal row filter. WHERE is faster for that.

Related: Compare with WHERE (next topic).

WHERE vs HAVING

Definition: Both filter data, but at different steps. WHERE filters rows. HAVING filters groups.

Simple Explanation: This is one of the most asked SQL questions. WHERE works before GROUP BY. HAVING works after GROUP BY.

Example: Total sales per city, only for 2024, and only cities with total above 1 lakh.

```
SELECT city, SUM(amount) AS total
FROM orders
WHERE order_date >= '2024-01-01'   -- row filter
GROUP BY city
HAVING SUM(amount) > 100000;     -- group filter
```

Technique (Step-by-Step): 1. Ask: am I filtering single rows or whole groups? 2. Single rows: use WHERE. 3. Groups (using SUM, COUNT, AVG): use HAVING. 4. You can use both in one query.

Interview Answer: “WHERE filters individual rows before grouping, and HAVING filters the groups after grouping. WHERE cannot use aggregates, but HAVING can.”

Follow-up Qs:
- **Q:** Which is faster? **A:** WHERE, because it removes rows early, before grouping.
- **Q:** Can one query have both? **A:** Yes. WHERE runs first, then GROUP BY, then HAVING.

Common Mistake: Putting an aggregate condition in WHERE, which gives an error.

Related: Linked to the order of execution (next topic).

Order of Execution

Definition: This is the order in which SQL runs the parts of a query. It is not the order you write them.

Simple Explanation: You write SELECT first, but SQL runs it almost last. This explains why you cannot use a SELECT alias in WHERE.

Example: Why does this fail? SELECT salary*12 AS yearly FROM employees WHERE yearly > 500000;

```
Written order : SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT
Run order     : FROM -> WHERE -> GROUP BY -> HAVING
                -> SELECT -> DISTINCT -> ORDER BY -> LIMIT
```

Technique (Step-by-Step): 1. Remember the run order: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT. 2. WHERE runs before SELECT, so it cannot see aliases. 3. ORDER BY runs after SELECT, so it can use aliases.

Interview Answer: “SQL runs FROM first, then WHERE, GROUP BY, HAVING, SELECT, ORDER BY and LIMIT. Knowing this helps me understand why aliases work in ORDER BY but not in WHERE.”

Follow-up Qs:
- **Q:** Can ORDER BY use a SELECT alias? **A:** Yes, because it runs after SELECT.
- **Q:** Can WHERE use a SELECT alias? **A:** No, because WHERE runs before SELECT.

Common Mistake: Using a column alias inside WHERE.

Related: Explains WHERE vs HAVING.

Level 2 - Quick Revision

Item	Key point
COUNT(*)	Counts all rows
COUNT(col)	Skips NULLs
SUM / AVG	Skip NULLs
MIN / MAX	Smallest / largest value
GROUP BY	One result per group
HAVING	Filters groups (can use aggregates)
WHERE	Filters rows (before grouping)

Key rule: Run order is FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT.

Top Interview Questions: (1) WHERE vs HAVING. (2) COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column). (3) Why can't you use an alias in WHERE?

Practical Scenarios: (1) Find customers with more than 5 orders - GROUP BY + HAVING. (2) Total sales per month - GROUP BY. (3) Find the highest order value per customer - MAX + GROUP BY.

LEVEL 3 - JOINS

INNER JOIN

Definition: INNER JOIN returns only the rows that have a match in both tables.

Simple Explanation: If a row has no match on the other side, it is dropped. It is the default join. Think of the middle part of two overlapping circles.

Example: Show each order with the customer name.

```
SELECT o.order_id, c.name, o.amount
FROM orders o
INNER JOIN customers c
  ON o.customer_id = c.customer_id;
```

Technique (Step-by-Step): 1. Write the first table in FROM and give it a short alias (o). 2. Write INNER JOIN and the second table with an alias (c). 3. Write ON and the matching columns. 4. In SELECT, use alias.column to avoid confusion.

Interview Answer: “INNER JOIN gives only the rows that match in both tables. I use it when I need data that must exist on both sides, like orders with valid customers.”

Follow-up Qs:
- **Q:** What happens to unmatched rows? **A:** They are removed from the result.
- **Q:** Is JOIN the same as INNER JOIN? **A:** Yes. JOIN alone means INNER JOIN.

Common Mistake: Forgetting the ON condition, which can create a huge wrong result.

Related: Compare with LEFT JOIN.

LEFT JOIN

Definition: LEFT JOIN returns all rows from the left table and the matching rows from the right table. If no match, the right side shows NULL.

Simple Explanation: Use it when you want to keep every row from the main table, even if there is no related data.

Example: Show all customers and their orders. Customers with no orders should still appear.

```
SELECT c.name, o.order_id
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id;
```

Technique (Step-by-Step): 1. Put the table you want to keep fully on the left (in FROM). 2. Write LEFT JOIN with the other table. 3. Write the ON condition. 4. Unmatched rows show NULL for right-table columns.

Interview Answer: “LEFT JOIN keeps all rows from the left table and adds matching data from the right. If there is no match, I get NULLs. I use it a lot to find missing data.”

Follow-up Qs:
- **Q:** LEFT JOIN vs INNER JOIN? **A:** INNER drops unmatched rows. LEFT keeps all left rows.
- **Q:** Where do you put filters on the right table? **A:** In the ON clause. If you put it in WHERE, the LEFT JOIN turns into an INNER JOIN.

Common Mistake: Filtering the right table in WHERE, which removes the NULL rows.

Related: Leads to the anti-join (finding unmatched rows).

RIGHT JOIN

Definition: RIGHT JOIN returns all rows from the right table and the matching rows from the left table.

Simple Explanation: It is the mirror of LEFT JOIN. Most people just swap the table order and use LEFT JOIN, because it is easier to read.

Example: Show all orders, even if the customer record is missing.

```
SELECT c.name, o.order_id
FROM customers c
RIGHT JOIN orders o
  ON c.customer_id = o.customer_id;
```

Technique (Step-by-Step): 1. Decide which table must be fully kept. 2. Put it on the right side of the JOIN. 3. Or swap the tables and use LEFT JOIN, which gives the same result.

Interview Answer: “RIGHT JOIN keeps all rows from the right table. It is the mirror of LEFT JOIN, so in practice I usually rewrite it as a LEFT JOIN.”

Follow-up Qs:
- **Q:** Why is RIGHT JOIN rarely used? **A:** You can swap the tables and use LEFT JOIN. It is easier to read.
- **Q:** Can they give the same result? **A:** Yes, if you swap the table order.

Common Mistake: Mixing up left and right when the query has many joins.

Related: Mirror of LEFT JOIN.

FULL OUTER JOIN

Definition: FULL OUTER JOIN returns all rows from both tables. Where there is no match, the missing side shows NULL.

Simple Explanation: It is LEFT JOIN and RIGHT JOIN together. Use it to compare two lists and see what is in one, in the other, or in both. MySQL does not support it directly.

Example: Compare two student lists: who is in list A, list B, or both.

```
SELECT a.id, b.id
FROM list_a a
FULL OUTER JOIN list_b b
  ON a.id = b.id;

-- MySQL workaround: LEFT JOIN UNION RIGHT JOIN
```

Technique (Step-by-Step): 1. Write FULL OUTER JOIN between the two tables. 2. Add the ON condition. 3. Rows with NULL on one side exist only in the other table. 4. In MySQL, combine a LEFT JOIN and a RIGHT JOIN with UNION.

Interview Answer: “FULL OUTER JOIN returns everything from both tables and fills NULL where there is no match. I use it for data reconciliation, like comparing two systems.”

Follow-up Qs:
- **Q:** Does MySQL support it? **A:** No. You can use LEFT JOIN UNION RIGHT JOIN.
- **Q:** Real use case? **A:** Comparing two data sources to find mismatches.

Common Mistake: Expecting it to work in MySQL without a workaround.

Related: Combines LEFT and RIGHT JOIN.

CROSS JOIN

Definition: CROSS JOIN pairs every row of the first table with every row of the second table.

Simple Explanation: There is no ON condition. If table A has 3 rows and table B has 4 rows, you get 12 rows. Use it to create all combinations.

Example: Make all combinations of sizes and colors.

```
SELECT s.size, c.color
FROM sizes s
CROSS JOIN colors c;
```

Technique (Step-by-Step): 1. Write CROSS JOIN with the second table. 2. Do not write ON. 3. Check the row count: rows in A multiplied by rows in B.

Interview Answer: “CROSS JOIN gives all possible combinations of rows from two tables. I use it to build things like a product and size grid, or a calendar for every store.”

Follow-up Qs:
- **Q:** How many rows does it return? **A:** Rows in table A times rows in table B.
- **Q:** When is it dangerous? **A:** On big tables, because the result becomes huge.

Common Mistake: Creating a CROSS JOIN by accident when the ON condition is missing.

Related: Related to the Cartesian product.

SELF JOIN

Definition: A SELF JOIN joins a table to itself.

Simple Explanation: You use two different aliases for the same table. It is used for data where one row points to another row in the same table, like employee and manager.

Example: Show each employee with their manager's name.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.emp_id;
```

Technique (Step-by-Step): 1. Write the same table twice with two aliases (e and m). 2. Think of e as the employee copy and m as the manager copy. 3. Join e.manager_id to m.emp_id. 4. Use LEFT JOIN to keep the top boss, who has no manager.

Interview Answer: “A self join is when a table joins to itself using two aliases. I use it for hierarchy data, like employees and their managers.”

Follow-up Qs:
- **Q:** Is SELF JOIN a special keyword? **A:** No. It is a normal join where both sides are the same table.
- **Q:** Why LEFT JOIN here? **A:** To keep employees who have no manager, like the CEO.

Common Mistake: Using the same alias for both copies of the table.

Related: Used in 'employees earning more than manager' (Level 11).

Anti-Join (Find Unmatched Rows)

Definition: An anti-join finds rows in one table that have no match in another table.

Simple Explanation: The most common way is LEFT JOIN with a WHERE ... IS NULL check. It answers questions like 'customers who never ordered'.

Example: Find customers who never placed an order.

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Technique (Step-by-Step): 1. LEFT JOIN the main table to the other table. 2. In WHERE, check that a column from the right table IS NULL. 3. Use a column that is never NULL when matched, like the primary key. 4. Another way is NOT EXISTS.

Interview Answer: “To find unmatched rows, I use a LEFT JOIN and then filter where the right table's key IS NULL. For example, customers who never placed an order.”

Follow-up Qs:
- **Q:** Other ways to do this? **A:** NOT EXISTS or NOT IN. NOT EXISTS is safer than NOT IN when NULLs are possible.
- **Q:** Why check the primary key for NULL? **A:** A real match can never have a NULL primary key, so NULL means no match.

Common Mistake: Checking a column that can be NULL even when matched.

Related: Uses LEFT JOIN; compare with NOT EXISTS (Level 4).

Level 3 - Quick Revision

Join	Returns
INNER	Only matching rows
LEFT	All left + matching right
RIGHT	All right + matching left
FULL OUTER	All rows from both
CROSS	Every combination
SELF	Table joined to itself
Anti-join	LEFT JOIN + IS NULL (unmatched rows)

Key rule: Filter the right table of a LEFT JOIN in ON, not in WHERE.

Top Interview Questions: (1) INNER vs LEFT vs FULL OUTER JOIN. (2) Find customers who never ordered. (3) What is a self join and when do you use it?

Practical Scenarios: (1) List all products and their sales, even products never sold - LEFT JOIN. (2) Employee and manager names - SELF JOIN. (3) Compare two customer lists from two systems - FULL OUTER JOIN.

LEVEL 4 - SUBQUERIES AND CTEs

Subquery

Definition: A subquery is a query written inside another query.

Simple Explanation: The inner query runs first and gives a value or a list. The outer query then uses it. You can use subqueries in WHERE, FROM and SELECT.

Example: Show employees who earn more than the average salary.

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Technique (Step-by-Step): 1. Write the inner query alone and test it. 2. Put it inside brackets. 3. Use it in WHERE with =, >, IN, or EXISTS. 4. If the inner query returns many rows, use IN, not =.

Interview Answer: “A subquery is a query inside another query. I use it when I need the result of one query as a filter for another, like comparing salary to the company average.”

Follow-up Qs:
- **Q:** Where can a subquery be used? **A:** In WHERE, in FROM (as a derived table) and in SELECT.
- **Q:** What if it returns many rows with =? **A:** You get an error. Use IN instead.

Common Mistake: Using = with a subquery that returns more than one row.

Related: Leads to correlated subquery and CTE.

Correlated Subquery

Definition: A correlated subquery uses a column from the outer query, so it runs once for each outer row.

Simple Explanation: A normal subquery runs once. A correlated one runs again for every row, so it can be slow on big tables.

Example: Find employees who earn more than the average of their own department.

```
SELECT e.name, e.salary, e.dept
FROM employees e
WHERE e.salary > (
    SELECT AVG(salary) FROM employees
    WHERE dept = e.dept);
```

Technique (Step-by-Step): 1. Write the outer query with an alias (e). 2. In the inner query, link to the outer row using e.column. 3. Test with a few rows. 4. If it is slow, try a JOIN or a window function.

Interview Answer: “A correlated subquery depends on the outer query, so it runs for each row. It is useful for per-group comparisons, but I prefer window functions or joins when speed matters.”

Follow-up Qs:
- **Q:** Correlated vs normal subquery? **A:** Normal runs once. Correlated runs once per outer row.
- **Q:** How to make it faster? **A:** Rewrite with a JOIN or a window function.

Common Mistake: Using a correlated subquery on a huge table without checking speed.

Related: Alternative: window functions (Level 5).

EXISTS vs IN

Definition: Both check if a value is found in another result. EXISTS checks if any row exists. IN checks if a value is in a list.

Simple Explanation: EXISTS stops as soon as it finds one match, so it is often faster on large data. NOT EXISTS is also safer than NOT IN when NULLs are present.

Example: Find customers who have placed at least one order.

```
SELECT c.name FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.customer_id);

-- same with IN
SELECT name FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders);
```

Technique (Step-by-Step): 1. For a yes/no check on related rows, use EXISTS. 2. Write SELECT 1 inside, since the data does not matter. 3. Link inner and outer with the key. 4. For 'not found', use NOT EXISTS.

Interview Answer: “IN compares a value against a list, and EXISTS checks if any matching row exists. I prefer NOT EXISTS over NOT IN, because NOT IN fails when the list contains NULL.”

Follow-up Qs:
- **Q:** Which is faster? **A:** EXISTS is often faster for large inner results, because it stops at the first match. Modern databases often optimize both.
- **Q:** Why is NOT IN risky? **A:** If the list has a NULL, NOT IN returns no rows.

Common Mistake: Using NOT IN on a column that has NULLs.

Related: Related to anti-join (Level 3).

CTE (WITH clause)

Definition: A CTE (Common Table Expression) is a named temporary result that you define with WITH and use in the main query.

Simple Explanation: It makes long queries easy to read. You can think of it as a temporary table that lives only for one query. You can define many CTEs in one query.

Example: Find departments whose total salary is above 5 lakh.

```
WITH dept_total AS (
    SELECT dept, SUM(salary) AS total
    FROM employees
    GROUP BY dept
)
SELECT dept, total
FROM dept_total
WHERE total > 500000;
```

Technique (Step-by-Step): 1. Write WITH and give the CTE a name. 2. Write its query inside brackets. 3. Write the main SELECT that uses the CTE name like a table. 4. For many CTEs, separate them with commas (write WITH only once).

Interview Answer: “A CTE is a named temporary result I define with WITH. I use it to break a complex query into clear steps, which is easier to read and fix than deeply nested subqueries.”

Follow-up Qs:
- **Q:** CTE vs subquery? **A:** They often give the same result. A CTE is more readable and can be used many times in the query.
- **Q:** Does a CTE store data permanently? **A:** No. It exists only during that one query.

Common Mistake: Writing WITH again for the second CTE instead of using a comma.

Related: Leads to recursive CTE.

Recursive CTE

Definition: A recursive CTE is a CTE that calls itself, to walk through levels of data like a tree.

Simple Explanation: It has two parts joined by UNION ALL: a starting query (anchor) and a part that repeats (recursive). It is used for hierarchies, like employee to manager chains.

Example: List all employees under manager 1, with their level.

```
WITH RECURSIVE org AS (
    SELECT emp_id, name, manager_id, 1 AS lvl
    FROM employees WHERE emp_id = 1
    UNION ALL
    SELECT e.emp_id, e.name, e.manager_id, o.lvl + 1
    FROM employees e
    JOIN org o ON e.manager_id = o.emp_id
)
SELECT * FROM org;
```

Technique (Step-by-Step): 1. Write the anchor query: the starting row(s). 2. Add UNION ALL. 3. Write the recursive query that joins the table to the CTE itself. 4. Make sure it will stop, when no new rows are found. 5. SQL Server uses WITH without the word RECURSIVE.

Interview Answer: “A recursive CTE repeats itself to go through hierarchy data, like finding everyone who reports to a manager at any level. It has an anchor part and a recursive part joined by UNION ALL.”

Follow-up Qs:

- **Q:** What are the two parts? **A:** The anchor (start) query and the recursive query.
- **Q:** How does it stop? **A:** It stops when the recursive part returns no new rows.

Common Mistake: Creating an endless loop because the data has a circular link.

Related: Builds on CTE and SELF JOIN.

Level 4 - Quick Revision

Topic	Key point
Subquery	Query inside a query; runs once
Correlated subquery	Uses outer row; runs per row
EXISTS	Yes/no check; stops at first match
IN	Value in a list
CTE	Named temporary result using WITH
Recursive CTE	Anchor + UNION ALL + recursive part

Key rule: Prefer NOT EXISTS over NOT IN when NULLs are possible.

Top Interview Questions: (1) EXISTS vs IN. (2) What is a CTE and why use it? (3) Correlated vs non-correlated subquery.

Practical Scenarios: (1) Employees earning above their department average - correlated subquery or window function. (2) Show an employee reporting chain - recursive CTE. (3) Break a long report query into steps - CTE.

LEVEL 5 - WINDOW FUNCTIONS

PARTITION BY and OVER

Definition: A window function does a calculation across related rows, but keeps every row in the result.

Simple Explanation: OVER() defines the window. PARTITION BY splits rows into groups (like GROUP BY, but it does not collapse rows). ORDER BY inside OVER sets the order within each group.

Example: Show each employee's salary and the average salary of their department, on the same row.

```
SELECT name, dept, salary,
       AVG(salary) OVER (PARTITION BY dept) AS dept_avg
FROM employees;
```

Technique (Step-by-Step): 1. Pick the function (AVG, SUM, ROW_NUMBER...). 2. Write OVER( ). 3. Inside, add PARTITION BY column to make groups. 4. Add ORDER BY inside if order matters. 5. Window functions can only be used in SELECT and ORDER BY.

Interview Answer: “A window function calculates across a group of rows without collapsing them. PARTITION BY makes the groups, and I keep all my original rows, which GROUP BY cannot do.”

Follow-up Qs:
- **Q:** GROUP BY vs PARTITION BY? **A:** GROUP BY collapses rows into one per group. PARTITION BY keeps all rows.
- **Q:** Can you use window functions in WHERE? **A:** No. Put them in a CTE or subquery first, then filter.

Common Mistake: Trying to filter on a window function directly in WHERE.

Related: Base for all topics in this level.

ROW_NUMBER

Definition: ROW_NUMBER gives a unique number to each row, 1, 2, 3, and so on, based on an order.

Simple Explanation: Even if two rows have the same value, they get different numbers. It is used to pick one row per group or to remove duplicates.

Example: Number employees from highest salary to lowest in each department.

```
SELECT name, dept, salary,
       ROW_NUMBER() OVER (PARTITION BY dept
                          ORDER BY salary DESC) AS rn
FROM employees;
```

Technique (Step-by-Step): 1. Write ROW_NUMBER() OVER (. 2. Add PARTITION BY for groups. 3. Add ORDER BY to decide the numbering. 4. Wrap in a CTE and filter rn = 1 to get the top row per group.

Interview Answer: “ROW_NUMBER gives each row a unique number in a chosen order. I use it to pick the latest record per customer and to remove duplicates.”

Follow-up Qs:
- **Q:** What happens on ties? **A:** Ties still get different numbers. The choice between tied rows is arbitrary.
- **Q:** Best use? **A:** Picking one row per group, and deleting duplicates.

Common Mistake: Using ROW_NUMBER when ties should get the same rank (use RANK or DENSE_RANK).

Related: Compare with RANK and DENSE_RANK.

RANK

Definition: RANK gives the same number to tied rows, and then skips the next numbers.

Simple Explanation: If two people tie for rank 2, both get 2, and the next person gets 4. There is a gap.

Example: Rank employees by salary. Salaries: 90, 80, 80, 70.

```
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees;
-- ranks: 1, 2, 2, 4
```

Technique (Step-by-Step): 1. Write RANK() OVER (ORDER BY column DESC). 2. Ties get the same rank. 3. The next rank skips numbers (1, 2, 2, 4).

Interview Answer: “RANK gives equal rows the same rank but leaves a gap after ties, like 1, 2, 2, 4. It is like ranking in a race.”

Follow-up Qs:
- **Q:** What are the ranks for 90, 80, 80, 70? **A:** 1, 2, 2, 4.
- **Q:** When to use RANK? **A:** When gaps after ties are fine, like sports rankings.

Common Mistake: Using RANK for Nth highest salary, which can skip the Nth number.

Related: Compare with DENSE_RANK.

DENSE_RANK

Definition: DENSE_RANK gives the same number to tied rows, and does not skip numbers.

Simple Explanation: For salaries 90, 80, 80, 70 the ranks are 1, 2, 2, 3. No gap. This is the best choice for Nth highest value problems.

Example: Find the 2nd highest salary.

```
SELECT DISTINCT salary FROM (
  SELECT salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) t
WHERE rnk = 2;
```

Technique (Step-by-Step): 1. Write DENSE_RANK() OVER (ORDER BY column DESC) in a subquery or CTE. 2. In the outer query, filter rnk = N. 3. Use DISTINCT if many people share the salary.

Interview Answer: “DENSE_RANK gives ties the same rank and does not leave gaps, like 1, 2, 2, 3. It is the right choice for Nth highest salary questions.”

Follow-up Qs:
- **Q:** ROW_NUMBER vs RANK vs DENSE_RANK? **A:** ROW_NUMBER is always unique. RANK has gaps after ties. DENSE_RANK has no gaps.
- **Q:** Which for Nth highest? **A:** DENSE_RANK.

Common Mistake: Mixing up RANK and DENSE_RANK in Nth highest problems.

Related: Used in Level 11 classic problems.

LAG and LEAD

Definition: LAG looks at the previous row's value. LEAD looks at the next row's value.

Simple Explanation: They let you compare a row with the row before or after it, without a self join. Common for change from last month.

Example: Show each month's revenue and last month's revenue.

```
SELECT month, revenue,
       LAG(revenue, 1) OVER (ORDER BY month) AS prev_rev,
       LEAD(revenue, 1) OVER (ORDER BY month) AS next_rev
FROM monthly_sales;
```

Technique (Step-by-Step): 1. Write LAG(column, 1) OVER (ORDER BY date_column). 2. The number 1 means one row back. Use 2 for two rows back. 3. The first row has no previous row, so it gives NULL. Add a third value for a default, like LAG(revenue, 1, 0). 4. Use LEAD for the next row.

Interview Answer: “LAG gives me the previous row's value and LEAD gives the next row's. I use LAG to calculate month-over-month change without a self join.”

Follow-up Qs:
- **Q:** What does LAG return on the first row? **A:** NULL, unless you give a default value.
- **Q:** Why better than self join? **A:** It is shorter, easier to read and usually faster.

Common Mistake: Forgetting ORDER BY inside OVER, so the previous row is random.

Related: Used for month-over-month growth (Level 11).

Running Total (SUM OVER)

Definition: A running total adds up values row by row, so each row shows the total so far.

Simple Explanation: Use SUM() OVER with ORDER BY. The same idea with a frame of the last N rows gives a moving average.

Example: Show a running total of daily sales, and a 3-day moving average.

```
SELECT sale_date, amount,
       SUM(amount) OVER (ORDER BY sale_date) AS running_total,
       AVG(amount) OVER (ORDER BY sale_date
           ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS moving_avg_3
FROM daily_sales;
```

Technique (Step-by-Step): 1. Write SUM(column) OVER (ORDER BY date). 2. Add PARTITION BY if each group needs its own total. 3. For a moving average, add ROWS BETWEEN 2 PRECEDING AND CURRENT ROW. 4. Check the first rows by hand.

Interview Answer: “A running total is SUM with OVER and ORDER BY, so each row shows the total up to that row. For a moving average I add a frame like ROWS BETWEEN 2 PRECEDING AND CURRENT ROW.”

Follow-up Qs:
- **Q:** How to restart the total for each group? **A:** Add PARTITION BY inside OVER.
- **Q:** What is a frame? **A:** It tells SQL which rows around the current row to include, like the last 3 rows.

Common Mistake: Leaving out ORDER BY, which gives the full total on every row.

Related: Related to LAG and growth reports.

NTILE

Definition: NTILE splits ordered rows into N equal-sized groups and gives each row a group number.

Simple Explanation: It is used to make buckets like quartiles (4 groups) or top 10 percent (10 groups).

Example: Split customers into 4 groups by total spend.

```
SELECT customer_id, total_spend,
       NTILE(4) OVER (ORDER BY total_spend DESC) AS quartile
FROM customer_spend;
```

Technique (Step-by-Step): 1. Write NTILE(N) OVER (ORDER BY column). 2. N is the number of groups. 3. Group 1 has the highest values when you sort DESC.

Interview Answer: “NTILE splits rows into N equal buckets. I use it for customer segmentation, like top 25 percent spenders.”

Follow-up Qs:
- **Q:** NTILE(4) gives what? **A:** Four groups, called quartiles.
- **Q:** What if rows do not divide evenly? **A:** The first groups get one extra row.

Common Mistake: Assuming each group has exactly the same size when rows do not divide evenly.

Related: Related to RANK.

Level 5 - Quick Revision

Function	Behavior
ROW_NUMBER	Unique 1,2,3 - no ties
RANK	Ties same, gap after (1,2,2,4)
DENSE_RANK	Ties same, no gap (1,2,2,3)
LAG / LEAD	Previous / next row value
SUM() OVER	Running total
NTILE(n)	Split into n buckets
PARTITION BY	Groups without collapsing rows

Key rule: Window functions cannot go in WHERE. Use a CTE, then filter.

Top Interview Questions: (1) ROW_NUMBER vs RANK vs DENSE_RANK. (2) GROUP BY vs PARTITION BY. (3) How do you find the top 3 salaries in each department?

Practical Scenarios: (1) Top 3 products by sales per category - DENSE_RANK + PARTITION BY. (2) Month-over-month growth - LAG. (3) Running total of sales per customer - SUM OVER.

LEVEL 6 - SET OPERATIONS AND CONDITIONAL LOGIC

UNION and UNION ALL

Definition: UNION joins the results of two queries and removes duplicates. UNION ALL joins them and keeps duplicates.

Simple Explanation: Both queries must have the same number of columns and similar data types. UNION ALL is faster because it does not check for duplicates.

Example: Combine customer lists from India and the US.

```
SELECT name, city FROM customers_india
UNION ALL
SELECT name, city FROM customers_us;
```

Technique (Step-by-Step): 1. Write the first SELECT. 2. Write UNION or UNION ALL. 3. Write the second SELECT with the same number of columns in the same order. 4. Put ORDER BY only once at the very end.

Interview Answer: “UNION combines two result sets and removes duplicates, while UNION ALL keeps everything and is faster. I use UNION ALL when I know there are no duplicates or I want them.”

Follow-up Qs:
- **Q:** UNION vs UNION ALL? **A:** UNION removes duplicates and is slower. UNION ALL keeps duplicates and is faster.
- **Q:** UNION vs JOIN? **A:** UNION stacks rows on top of each other. JOIN adds columns side by side.

Common Mistake: Using UNION when UNION ALL is enough, which wastes time.

Related: Related to INTERSECT and EXCEPT.

INTERSECT

Definition: INTERSECT returns only the rows that appear in both query results.

Simple Explanation: It is like the overlap of two lists. Duplicates are removed.

Example: Find customers who bought in both 2023 and 2024.

```
SELECT customer_id FROM orders_2023
INTERSECT
SELECT customer_id FROM orders_2024;
```

Technique (Step-by-Step): 1. Write two SELECT queries with the same columns. 2. Put INTERSECT between them. 3. In MySQL (older versions), use INNER JOIN or IN instead.

Interview Answer: “INTERSECT gives the rows that are common to both queries. I use it to find things like customers who are active in two different years.”

Follow-up Qs:
- **Q:** Alternative if not supported? **A:** Use INNER JOIN or EXISTS.
- **Q:** Does it remove duplicates? **A:** Yes.

Common Mistake: Using it in a database version that does not support it.

Related: Related to UNION and EXCEPT.

EXCEPT / MINUS

Definition: EXCEPT returns rows from the first query that are not in the second query. Oracle calls it MINUS.

Simple Explanation: Order matters here. A EXCEPT B is not the same as B EXCEPT A.

Example: Find customers who bought in 2023 but not in 2024.

```
SELECT customer_id FROM orders_2023
EXCEPT
SELECT customer_id FROM orders_2024;
```

Technique (Step-by-Step): 1. Write the main list first. 2. Put EXCEPT (or MINUS in Oracle). 3. Write the list to remove. 4. Alternative: LEFT JOIN with IS NULL, or NOT EXISTS.

Interview Answer: “EXCEPT shows what is in the first result but not in the second. I use it to find lost customers, like those who bought last year but not this year.”

Follow-up Qs:
- **Q:** Does the order of queries matter? **A:** Yes. A EXCEPT B is different from B EXCEPT A.
- **Q:** Other name? **A:** MINUS in Oracle.

Common Mistake: Writing the two queries in the wrong order.

Related: Similar idea to anti-join.

CASE WHEN

Definition: CASE WHEN is SQL's if-then-else. It returns different values based on conditions.

Simple Explanation: SQL checks conditions from top to bottom and returns the result of the first true one. ELSE handles everything else. It is used for labels, buckets and conditional sums.

Example: Label employees as High, Medium or Low by salary.

```
SELECT name, salary,
  CASE
    WHEN salary >= 100000 THEN 'High'
    WHEN salary >= 50000  THEN 'Medium'
    ELSE 'Low'
  END AS salary_band
FROM employees;
```

Technique (Step-by-Step): 1. Write CASE. 2. Add WHEN condition THEN result for each case. 3. Order the conditions carefully, because the first match wins. 4. Add ELSE for the rest. 5. End with END and an alias. 6. Use it inside SUM for conditional totals.

Interview Answer: “CASE WHEN is how I write if-else logic in SQL. I use it to create labels like salary bands, and inside SUM to count or total only rows that meet a condition.”

Follow-up Qs:
- **Q:** What if no WHEN is true and there is no ELSE? **A:** It returns NULL.
- **Q:** Can you use CASE inside SUM? **A:** Yes. SUM(CASE WHEN status = 'Paid' THEN amount ELSE 0 END) gives the total of paid orders only.

Common Mistake: Putting the conditions in the wrong order, so a broad one catches everything first.

Related: Used for pivoting data and bucketing.

COALESCE

Definition: COALESCE returns the first value that is not NULL from a list.

Simple Explanation: Use it to replace NULL with a default value, like 0 or 'Unknown'.

Example: Show 'No Phone' when a phone number is missing.

```
SELECT name, COALESCE(phone, 'No Phone') AS phone
FROM customers;
```

Technique (Step-by-Step): 1. Write COALESCE(. 2. List the column first, then the fallback value. 3. You can add many fallbacks, like COALESCE(mobile, home, 'None'). 4. Keep the same data type for all values.

Interview Answer: “COALESCE gives the first non-NULL value in a list. I use it to replace NULLs with a default, like turning a missing sales number into 0 before a calculation.”

Follow-up Qs:
- **Q:** COALESCE vs IFNULL? **A:** IFNULL takes only two values and is MySQL-specific. COALESCE takes many and works in all databases.
- **Q:** Can it be used in calculations? **A:** Yes. Use it to avoid NULL breaking a sum or division.

Common Mistake: Mixing data types, like text and number, inside COALESCE.

Related: Related to IS NULL (Level 1).

NULLIF

Definition: NULLIF returns NULL if two values are equal. Otherwise it returns the first value.

Simple Explanation: Its best use is to avoid a divide-by-zero error. If the divisor is 0, NULLIF turns it into NULL, and the division returns NULL instead of an error.

Example: Calculate profit margin without a divide-by-zero error.

```
SELECT profit / NULLIF(revenue, 0) AS margin
FROM sales;
```

Technique (Step-by-Step): 1. Find the value that can cause a problem, like 0 in a divisor. 2. Wrap it as NULLIF(column, 0). 3. Optionally wrap the full result in COALESCE to show 0.

Interview Answer: “NULLIF returns NULL when two values are equal. I mostly use it to stop divide-by-zero errors by writing NULLIF(divisor, 0).”

Follow-up Qs:
- **Q:** Why useful in division? **A:** It turns a 0 divisor into NULL, so there is no error.
- **Q:** NULLIF vs COALESCE? **A:** COALESCE replaces NULL with a value. NULLIF creates NULL from a value.

Common Mistake: Forgetting NULLIF and getting a divide-by-zero error in reports.

Related: Opposite idea to COALESCE.

Level 6 - Quick Revision

Item	Key point
UNION	Combine + remove duplicates
UNION ALL	Combine + keep duplicates (faster)
INTERSECT	Rows in both
EXCEPT / MINUS	Rows in first, not in second
CASE WHEN	If-then-else in SQL
COALESCE	First non-NULL value
NULLIF	Returns NULL if both values are equal

Key rule: Both queries in a set operation need the same number of columns and similar types.

Top Interview Questions: (1) UNION vs UNION ALL. (2) How do you avoid divide-by-zero? (3) COALESCE vs IFNULL.

Practical Scenarios: (1) Find customers who bought last year but not this year - EXCEPT. (2) Create age groups from an age column - CASE WHEN. (3) Replace missing sales values with 0 - COALESCE.

LEVEL 7 - STRING AND DATE FUNCTIONS

SUBSTRING / LEFT / RIGHT

Definition: These functions cut a part of a text value.

Simple Explanation: SUBSTRING takes text from any position. LEFT takes from the start. RIGHT takes from the end. Names differ a little in each database (SUBSTR in Oracle).

Example: Get the first 3 letters of a product code and the last 4 digits of a phone.

```
SELECT LEFT(product_code, 3)  AS prefix,
       RIGHT(phone, 4)        AS last4,
       SUBSTRING(code, 5, 4)  AS middle
FROM products;
```

Technique (Step-by-Step): 1. LEFT(text, n) gives the first n characters. 2. RIGHT(text, n) gives the last n characters. 3. SUBSTRING(text, start, length) starts from position start (counting begins at 1). 4. Use with position-finding functions (like POSITION or CHARINDEX) for flexible cuts.

Interview Answer: “SUBSTRING, LEFT and RIGHT help me cut parts of text, like getting the year out of an ID code. I check the exact function name for the database I am using.”

Follow-up Qs:
- **Q:** Does counting start at 0 or 1? **A:** At 1 in SQL.
- **Q:** Oracle name? **A:** SUBSTR.

Common Mistake: Starting the position at 0 instead of 1.

Related: Similar to Excel's LEFT/RIGHT/MID.

CONCAT

Definition: CONCAT joins two or more text values into one.

Simple Explanation: You can also use || (PostgreSQL, Oracle) or + (SQL Server). If any value is NULL, some databases return NULL, so use COALESCE.

Example: Make a full name from first and last name.

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM employees;

-- PostgreSQL style
SELECT first_name || ' ' || last_name FROM employees;
```

Technique (Step-by-Step): 1. Write CONCAT(. 2. List the values and add ' ' in quotes for a space. 3. Wrap nullable columns in COALESCE. 4. Or use CONCAT_WS(' ', a, b) to join with a separator.

Interview Answer: “CONCAT joins text values together, like first and last name. I add a space between them and handle NULLs with COALESCE.”

Follow-up Qs:
- **Q:** What happens with NULL? **A:** With || or + the result becomes NULL. MySQL's CONCAT also returns NULL.
- **Q:** What is CONCAT_WS? **A:** It joins values with a separator and skips NULLs.

Common Mistake: Forgetting the space between the values, giving 'JohnSmith'.

Related: Reverse of splitting text.

UPPER, LOWER and TRIM

Definition: UPPER makes text capital. LOWER makes text small. TRIM removes extra spaces at the start and end.

Simple Explanation: They are used to clean text before you compare or group it. 'Delhi', 'delhi ' and 'DELHI' look different to SQL until you clean them.

Example: Clean the city column so all values look the same.

```
SELECT UPPER(TRIM(city)) AS clean_city,
       COUNT(*) AS cnt
FROM customers
GROUP BY UPPER(TRIM(city));
```

Technique (Step-by-Step): 1. Wrap the column with TRIM to remove spaces. 2. Wrap that with UPPER or LOWER for one case. 3. Use the same clean expression in GROUP BY or JOIN. 4. Use LTRIM or RTRIM for one side only.

Interview Answer: “UPPER, LOWER and TRIM clean text so that values match properly. I always trim and fix the case before grouping or joining on a text column.”

Follow-up Qs:

- **Q:** Why does TRIM matter for joins? **A:** A hidden space makes two equal-looking values not match.
- **Q:** Does TRIM remove spaces in the middle? **A:** No. It removes only spaces at the start and end.

Common Mistake: Grouping on a messy text column without cleaning it first.

Related: Similar to Excel's TRIM and UPPER.

REPLACE and LENGTH

Definition: REPLACE swaps one piece of text for another. LENGTH (or LEN) counts the characters.

Simple Explanation: REPLACE is used to fix wrong characters. LENGTH is used to check data, like a phone number must have 10 digits.

Example: Remove dashes from phone numbers and find the ones that are not 10 digits.

```
SELECT REPLACE(phone, '-', '') AS clean_phone
FROM customers
WHERE LENGTH(REPLACE(phone, '-', '')) <> 10;
```

Technique (Step-by-Step): 1. Write REPLACE(text, 'old', 'new'). 2. To delete something, replace it with ''. 3. Use LENGTH(text) to count characters (LEN in SQL Server). 4. Use both together for data checks.

Interview Answer: “REPLACE fixes characters in text, and LENGTH counts them. I use them together for data quality checks, like confirming phone numbers have 10 digits.”

Follow-up Qs:

- **Q:** LENGTH vs LEN? **A:** LENGTH is in MySQL/PostgreSQL/Oracle. LEN is in SQL Server.
- **Q:** Does LENGTH count spaces? **A:** Yes.

Common Mistake: Forgetting that spaces count in the length.

Related: Used in data cleaning.

DATEDIFF and DATEADD

Definition: DATEDIFF finds the gap between two dates. DATEADD adds time to a date.

Simple Explanation: Syntax changes by database. SQL Server: DATEDIFF(day, start, end). MySQL: DATEDIFF(end, start). PostgreSQL: end - start, or date + INTERVAL.

Example: Find how many days each order took to ship, and a due date 7 days later.

```
-- MySQL
SELECT DATEDIFF(ship_date, order_date) AS days_taken,
       DATE_ADD(order_date, INTERVAL 7 DAY) AS due_date
FROM orders;

-- PostgreSQL
SELECT ship_date - order_date AS days_taken,
       order_date + INTERVAL '7 days' AS due_date
FROM orders;
```

Technique (Step-by-Step): 1. Check which database you are using. 2. For the gap, use DATEDIFF (or subtract dates in PostgreSQL). 3. To add time, use DATE_ADD or DATEADD or + INTERVAL. 4. Check the result with one row by hand.

Interview Answer: “DATEDIFF gives the gap between two dates and DATEADD adds time to a date. The syntax is different in each database, so I always check it first.”

Follow-up Qs:
- **Q:** Is the DATEDIFF argument order the same everywhere? **A:** No. MySQL uses (end, start). SQL Server uses (unit, start, end).
- **Q:** How to subtract days? **A:** Use a negative number or - INTERVAL.

Common Mistake: Mixing up the order of start and end date, giving a negative result.

Related: Similar to Excel's DATEDIF and WORKDAY.

EXTRACT and DATE_TRUNC

Definition: EXTRACT pulls one part (year, month, day) from a date. DATE_TRUNC cuts a date down to the start of a period, like the first day of the month.

Simple Explanation: Use them to group data by month or year. DATE_TRUNC is in PostgreSQL. MySQL uses DATE_FORMAT or YEAR() and MONTH().

Example: Total sales per month.

```
-- PostgreSQL
SELECT DATE_TRUNC('month', order_date) AS month,
       SUM(amount) AS revenue
FROM orders
GROUP BY 1
ORDER BY 1;

-- Get only the year
SELECT EXTRACT(YEAR FROM order_date) FROM orders;
```

Technique (Step-by-Step): 1. Pick the period: day, month or year. 2. Use DATE_TRUNC('month', date) to group by month. 3. Use EXTRACT(YEAR FROM date) for just the number. 4. In MySQL use YEAR(date) and MONTH(date).

Interview Answer: “I use DATE_TRUNC or EXTRACT to group by month or year in reports. DATE_TRUNC keeps a proper date, which sorts correctly.”

Follow-up Qs:
- **Q:** EXTRACT vs DATE_TRUNC? **A:** EXTRACT gives a number like 3. DATE_TRUNC gives a date like 2024-03-01.
- **Q:** Why group by month using DATE_TRUNC? **A:** It separates 2023-03 from 2024-03, but EXTRACT(MONTH) would mix them.

Common Mistake: Grouping by month number only, which mixes data from different years.

Related: Used in month-over-month growth (Level 11).

CURRENT_DATE and NOW

Definition: CURRENT_DATE gives today's date. NOW() or CURRENT_TIMESTAMP gives today's date and time.

Simple Explanation: They are used to filter recent data, like orders in the last 30 days, so the report always stays up to date.

Example: Show orders from the last 30 days.

```
-- PostgreSQL
SELECT * FROM orders
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days';

-- MySQL
SELECT * FROM orders
WHERE order_date >= CURDATE() - INTERVAL 30 DAY;
```

Technique (Step-by-Step): 1. Use CURRENT_DATE when time is not needed. 2. Use NOW() or CURRENT_TIMESTAMP when time matters. 3. Subtract an interval to look back. 4. Remember that the value

changes each day.

Interview Answer: “CURRENT_DATE gives today's date, so my queries always use fresh data, like the last 30 days, without changing the code.”

Follow-up Qs:
- **Q:** CURRENT_DATE vs NOW()? **A:** CURRENT_DATE gives only the date. NOW() gives date and time.
- **Q:** MySQL name for today? **A:** CURDATE().

Common Mistake: Using NOW() when comparing to a date column, and missing today's rows.

Related: Same idea as Excel's TODAY and NOW.

Level 7 - Quick Revision

Function	Purpose
SUBSTRING / LEFT / RIGHT	Cut part of text
CONCAT	Join text
UPPER / LOWER / TRIM	Clean text
REPLACE / LENGTH	Swap text / count characters
DATEDIFF / DATEADD	Gap between dates / add time
EXTRACT / DATE_TRUNC	Get date part / round date to period
CURRENT_DATE / NOW	Today / today with time

Key rule: Date and string function names change between MySQL, PostgreSQL and SQL Server. Always check.

Top Interview Questions: (1) How do you group sales by month? (2) How do you clean messy text before a join? (3) How do you find the days between two dates?

Practical Scenarios: (1) Clean city names with different spelling and spaces - UPPER + TRIM. (2) Orders in the last 30 days - CURRENT_DATE. (3) Average delivery time in days - DATEDIFF.

LEVEL 8 - DDL, DML AND CONSTRAINTS

CREATE TABLE

Definition: CREATE TABLE makes a new table with columns, data types and rules.

Simple Explanation: This is part of DDL (Data Definition Language), the commands that build the structure of the database.

Example: Create an employees table.

```
CREATE TABLE employees (
    emp_id     INT PRIMARY KEY,
    name       VARCHAR(100) NOT NULL,
    salary     DECIMAL(10,2) DEFAULT 0,
    manager_id INT,
    hire_date  DATE
);
```

Technique (Step-by-Step): 1. Write CREATE TABLE and the table name. 2. Inside brackets, list each column with its data type. 3. Add rules like PRIMARY KEY, NOT NULL, DEFAULT. 4. Separate columns with commas.

Interview Answer: “CREATE TABLE builds a new table. I choose the right data type for each column and add constraints like PRIMARY KEY and NOT NULL so the data stays clean.”

Follow-up Qs:

- **Q:** What is DDL? **A:** Data Definition Language: CREATE, ALTER, DROP, TRUNCATE.
- **Q:** Why choose data types carefully? **A:** It saves space, speeds up queries and stops wrong data.

Common Mistake: Using VARCHAR for dates or numbers.

Related: Leads to ALTER TABLE and constraints.

ALTER TABLE

Definition: ALTER TABLE changes the structure of an existing table.

Simple Explanation: You can add, drop or change a column, or add a constraint, without making the table again.

Example: Add a city column and then remove it.

```
ALTER TABLE employees ADD city VARCHAR(50);
ALTER TABLE employees DROP COLUMN city;
ALTER TABLE employees
  ADD CONSTRAINT chk_sal CHECK (salary >= 0);
```

Technique (Step-by-Step): 1. Write ALTER TABLE and the table name. 2. Choose the action: ADD, DROP COLUMN, MODIFY / ALTER COLUMN. 3. Test on a copy first for big or live tables. 4. Take a backup before dropping a column.

Interview Answer: “ALTER TABLE lets me change a table after it is made, such as adding a column or a constraint. On live tables I test first, because some changes can lock the table.”

Follow-up Qs:

- **Q:** Can ALTER TABLE lose data? **A:** Yes. Dropping a column deletes its data.
- **Q:** What can it add? **A:** Columns, constraints and indexes.

Common Mistake: Changing a live production table without a backup.

Related: Part of DDL.

INSERT

Definition: INSERT adds new rows to a table.

Simple Explanation: Always write the column names. If the table changes later, your insert will still work. You can insert many rows at once, or insert from a SELECT.

Example: Add two employees, and copy rich employees into another table.

```
INSERT INTO employees (emp_id, name, salary)
VALUES (1, 'Asha', 60000),
       (2, 'Ravi', 55000);

INSERT INTO high_paid (emp_id, name)
SELECT emp_id, name FROM employees
WHERE salary > 100000;
```

Technique (Step-by-Step): 1. Write INSERT INTO table (columns). 2. Write VALUES with the same number and order of values. 3. Add many rows by repeating (...) with commas. 4. Or use INSERT ... SELECT to copy data.

Interview Answer: “INSERT adds rows to a table. I always list the column names, and I use INSERT with SELECT when I need to copy data from another table.”

Follow-up Qs:
- **Q:** Why list the columns? **A:** The query will not break if the table gets a new column.
- **Q:** What is DML? **A:** Data Manipulation Language: SELECT, INSERT, UPDATE, DELETE.

Common Mistake: Inserting values without column names, in the wrong order.

Related: Part of DML.

UPDATE

Definition: UPDATE changes existing values in a table.

Simple Explanation: Always use WHERE. Without WHERE, every row in the table is updated.

Example: Give a 10 percent raise to employees in the Sales department.

```
UPDATE employees
SET salary = salary * 1.10
WHERE dept = 'Sales';
```

Technique (Step-by-Step): 1. First run a SELECT with the same WHERE to see which rows will change. 2. Write UPDATE table SET column = value. 3. Add the WHERE condition. 4. Run inside a transaction so you can ROLLBACK if wrong.

Interview Answer: “UPDATE changes existing data. I always write the SELECT first with the same WHERE to check the rows, and I use a transaction so I can undo a mistake.”

Follow-up Qs:
- **Q:** What if you forget WHERE? **A:** All rows are updated.
- **Q:** How do you stay safe? **A:** Test with SELECT first and use a transaction.

Common Mistake: Running UPDATE without WHERE.

Related: Leads to transactions (Level 9).

DELETE vs TRUNCATE vs DROP

Definition: DELETE removes selected rows. TRUNCATE removes all rows quickly. DROP removes the whole table.

Simple Explanation: This is a very common interview question. DELETE is DML and can use WHERE. TRUNCATE and DROP are DDL and cannot be filtered.

Example: Remove old orders, empty a log table, and remove a table.

```
DELETE FROM orders WHERE order_date < '2020-01-01';
TRUNCATE TABLE log_table;
DROP TABLE old_table;
```

Technique (Step-by-Step): 1. Need only some rows removed? Use DELETE with WHERE. 2. Need to empty the whole table fast and keep the structure? Use TRUNCATE. 3. Need the table gone completely? Use DROP. 4. Always back up first.

Interview Answer: “DELETE removes chosen rows and can be rolled back. TRUNCATE empties the whole table fast but keeps its structure. DROP removes the table completely.”

Follow-up Qs:
- **Q:** Which is fastest to empty a table? **A:** TRUNCATE.
- **Q:** Can DELETE be rolled back? **A:** Yes, inside a transaction. TRUNCATE rollback depends on the database; PostgreSQL and SQL Server allow it in a transaction, MySQL does not.

Common Mistake: Using DELETE without WHERE, or DROP on the wrong table.

Related: Linked to transactions.

Constraints

Definition: Constraints are rules on a table that keep the data correct.

Simple Explanation: Main constraints: NOT NULL (value required), UNIQUE (no repeats), PRIMARY KEY (unique and not null), FOREIGN KEY (must exist in another table), CHECK (must meet a condition), DEFAULT (value used when none is given).

Example: A table that follows all the rules.

```
CREATE TABLE orders (
    order_id    INT PRIMARY KEY,
    customer_id INT NOT NULL,
    amount      DECIMAL(10,2) CHECK (amount > 0),
    status      VARCHAR(20) DEFAULT 'New',
    FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
);
```

Technique (Step-by-Step): 1. Decide the rules for each column. 2. Add them in CREATE TABLE, or later with ALTER TABLE ADD CONSTRAINT. 3. Give constraints clear names. 4. Test by trying to insert wrong data.

Interview Answer: “Constraints are rules that protect data quality, like NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK and DEFAULT. They stop bad data at the door.”

Follow-up Qs:
- **Q:** PRIMARY KEY vs UNIQUE? **A:** A table has one primary key and it cannot be NULL. A table can have many UNIQUE columns and they can allow one NULL.
- **Q:** What does FOREIGN KEY do? **A:** It makes sure the value exists in the parent table.

Common Mistake: Skipping constraints and cleaning bad data later.

Related: Leads to keys and normalization (Level 9).

Level 8 - Quick Revision

Command	Purpose
CREATE / ALTER / DROP	Build or change structure (DDL)
INSERT / UPDATE / DELETE	Change data (DML)
DELETE	Remove chosen rows, can use WHERE
TRUNCATE	Empty table fast, keep structure
DROP	Remove the whole table
Constraints	NOT NULL, UNIQUE, PK, FK, CHECK, DEFAULT

Key rule: Never run UPDATE or DELETE without WHERE.

Top Interview Questions: (1) DELETE vs TRUNCATE vs DROP. (2) PRIMARY KEY vs UNIQUE. (3) DDL vs DML.

Practical Scenarios: (1) Add a new column to an existing table - ALTER TABLE. (2) Clear a staging table every night - TRUNCATE. (3) Stop negative prices - CHECK constraint.

LEVEL 9 - DATABASE DESIGN AND TRANSACTIONS

Keys (Primary, Foreign, Unique, Composite)

Definition: Keys are columns used to identify rows and link tables.

Simple Explanation: Primary key: unique id of a row. Foreign key: points to the primary key of another table. Unique key: no repeated values. Composite key: a key made from two or more columns together.

Example: customers(customer_id) is the primary key. orders(customer_id) is a foreign key to it.

```
CREATE TABLE order_items (
    order_id   INT,
    product_id INT,
    qty        INT,
    PRIMARY KEY (order_id, product_id)  -- composite key
);
```

Technique (Step-by-Step): 1. Choose one column that is unique for every row as the primary key. 2. In child tables, add a column with the same value as the parent key and make it a foreign key. 3. Use a composite key when one column alone is not unique.

Interview Answer: “A primary key identifies each row uniquely. A foreign key links to another table's primary key. A composite key uses more than one column together, like order_id and product_id in an order items table.”

Follow-up Qs:
- **Q:** Can a table have two primary keys? **A:** No. But one primary key can have many columns (composite).
- **Q:** Can a foreign key be NULL? **A:** Yes, unless you add NOT NULL.

Common Mistake: Choosing a column that can repeat, like name, as a primary key.

Related: Leads to normalization.

Normalization (1NF, 2NF, 3NF)

Definition: Normalization is organizing tables to remove repeated data and avoid mistakes.

Simple Explanation: 1NF: each cell has one value, no lists. 2NF: 1NF and every column depends on the whole key. 3NF: 2NF and no column depends on another non-key column. The result is smaller tables linked with keys.

Example: Do not store the customer city in every order row. Store it once in the customers table.

```
Bad : orders(order_id, customer_name, customer_city, amount)
Good: orders(order_id, customer_id, amount)
      customers(customer_id, name, city)
```

Technique (Step-by-Step): 1. Check for cells with many values and split them (1NF). 2. Check for columns that depend on only part of a composite key and move them out (2NF). 3. Check for columns that depend on another normal column and move them out (3NF).

Interview Answer: “Normalization splits data into related tables so each fact is stored once. This avoids update mistakes and saves space. I know 1NF, 2NF and 3NF.”

Follow-up Qs:
- **Q:** Why normalize? **A:** To avoid repeated data and update, insert and delete problems.
- **Q:** What is 3NF in one line? **A:** Every column depends on the key, the whole key and nothing but the key.

Common Mistake: Over-normalizing, which creates too many joins.

Related: Opposite: denormalization.

Denormalization

Definition: Denormalization means adding repeated data on purpose to make reads faster.

Simple Explanation: It is used in reporting and data warehouses, where fast reads matter more than avoiding repetition.

Example: Store customer_city inside the sales table so reports do not need a join.

```
sales(sale_id, customer_id, customer_city, amount)
```

Technique (Step-by-Step): 1. Find a slow report that needs many joins. 2. Copy the needed columns into one wide table. 3. Update this table on a schedule. 4. Accept that the data is repeated.

Interview Answer: “Denormalization means I keep some repeated data on purpose to make reports faster. It is common in data warehouses, where reading speed is more important than saving space.”

Follow-up Qs:
- **Q:** Normalization vs denormalization? **A:** Normalization reduces repeated data. Denormalization adds it back for speed.
- **Q:** When is it used? **A:** In reporting and analytics systems.

Common Mistake: Denormalizing a transaction system, which can cause wrong data.

Related: Opposite of normalization.

ACID Properties

Definition: ACID is a set of four promises that make database transactions reliable.

Simple Explanation: Atomicity: all or nothing. Consistency: data follows all rules before and after. Isolation: transactions do not disturb each other. Durability: saved data stays saved, even after a crash.

Example: A bank transfer: take 500 from A and add 500 to B. Both steps must happen, or neither.

```
BEGIN;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT;
```

Technique (Step-by-Step): 1. Remember the word ACID: Atomicity, Consistency, Isolation, Durability. 2. For each letter, remember one simple line, like 'all or nothing' for A. 3. Use the bank transfer example in the interview.

Interview Answer: “ACID makes transactions safe. Atomicity means all or nothing, consistency keeps rules true, isolation keeps transactions separate, and durability keeps committed data safe. A bank transfer is a good example.”

Follow-up Qs:
- **Q:** What does atomicity mean? **A:** All steps happen, or none happen.
- **Q:** What does durability mean? **A:** After COMMIT, data is safe even if the system crashes.

Common Mistake: Mixing up consistency and isolation.

Related: Leads to transactions.

Transactions (COMMIT and ROLLBACK)

Definition: A transaction is a group of SQL steps treated as one unit.

Simple Explanation: COMMIT saves all changes. ROLLBACK cancels all changes made since BEGIN. SAVEPOINT lets you roll back to a middle point.

Example: Update salaries, check the result, then undo if wrong.

```
BEGIN;
UPDATE employees SET salary = salary * 1.1 WHERE dept = 'Sales';
-- check with SELECT, then:
ROLLBACK;   -- or COMMIT;
```

Technique (Step-by-Step): 1. Write BEGIN (or START TRANSACTION). 2. Run your changes. 3. Check the result with SELECT. 4. Write COMMIT to save, or ROLLBACK to cancel.

Interview Answer: “A transaction groups steps so they succeed or fail together. I use BEGIN, then check my work, and finish with COMMIT or ROLLBACK. It protects me from half-finished changes.”

Follow-up Qs:
- **Q:** COMMIT vs ROLLBACK? **A:** COMMIT saves changes. ROLLBACK cancels them.

- **Q:** What is a savepoint? **A:** A marker inside a transaction that you can roll back to.

Common Mistake: Forgetting to COMMIT or ROLLBACK and leaving the transaction open, which can lock tables.

Related: Related to ACID.

Level 9 - Quick Revision

Topic	Key point
Primary key	Unique and not NULL; one per table
Foreign key	Links to another table's key
Composite key	Key made of 2+ columns
Normalization	Remove repeated data (1NF, 2NF, 3NF)
Denormalization	Add repeated data for speed
ACID	Atomicity, Consistency, Isolation, Durability
COMMIT / ROLLBACK	Save / cancel a transaction

Key rule: Normalize for transaction systems; denormalize for reporting systems.

Top Interview Questions: (1) What is normalization? Explain 1NF, 2NF, 3NF. (2) Explain ACID with an example. (3) Primary key vs foreign key vs unique key.

Practical Scenarios: (1) Design tables for an online store - keys and normalization. (2) Money transfer between two accounts - transaction with ACID. (3) Build a fast reporting table - denormalization.

LEVEL 10 - PERFORMANCE AND ADVANCED OBJECTS

Indexes (Clustered vs Non-Clustered)

Definition: An index is a lookup structure that helps the database find rows fast, like the index at the back of a book.

Simple Explanation: A clustered index decides the physical order of the rows, so a table has only one. A non-clustered index is a separate list that points to the rows, and a table can have many.

Example: Make searches by email fast.

```
CREATE INDEX idx_email
ON customers (email);

SELECT * FROM customers
WHERE email = 'a@x.com';   -- now fast
```

Technique (Step-by-Step): 1. Find the columns used often in WHERE, JOIN and ORDER BY. 2. Create an index on them with CREATE INDEX. 3. Do not index every column, because indexes slow down INSERT and UPDATE. 4. Check with EXPLAIN that the index is used.

Interview Answer: “An index helps the database find rows quickly without reading the whole table. I add indexes on columns used in filters and joins, but not too many, because they slow down writes.”

Follow-up Qs:
- **Q:** Clustered vs non-clustered? **A:** Clustered sets the actual row order, one per table. Non-clustered is a separate structure, many per table.
- **Q:** What is the downside of indexes? **A:** They use space and slow down INSERT, UPDATE and DELETE.

Common Mistake: Putting a function on an indexed column in WHERE, like WHERE YEAR(date) = 2024, which stops the index being used.

Related: Check with EXPLAIN.

EXPLAIN (Execution Plan)

Definition: EXPLAIN shows how the database will run your query: which tables it reads, which indexes it uses and in what order.

Simple Explanation: It is the main tool to find why a query is slow.

Example: See if a query uses an index.

```
EXPLAIN
SELECT * FROM orders
WHERE customer_id = 101;

-- PostgreSQL: EXPLAIN ANALYZE also runs the query and shows real time
```

Technique (Step-by-Step): 1. Write EXPLAIN before your query. 2. Look for 'Seq Scan' or 'Full Table Scan'. It means no index is used. 3. Look for 'Index Scan'. It is good. 4. Add an index or rewrite the query, then run EXPLAIN again.

Interview Answer: “EXPLAIN shows the plan the database will use for a query. I check it for full table scans, then add an index or rewrite the query to make it faster.”

Follow-up Qs:
- **Q:** What is a full table scan? **A:** The database reads every row of the table.
- **Q:** What does EXPLAIN ANALYZE do? **A:** It actually runs the query and shows real timings.

Common Mistake: Guessing the problem without looking at the plan.

Related: Works with indexes.

View vs Materialized View

Definition: A view is a saved query that acts like a virtual table. A materialized view is a saved query whose result is also stored on disk.

Simple Explanation: A view runs its query every time you use it, so it is always fresh. A materialized view is faster to read but must be refreshed, so it can be old.

Example: Save a daily sales summary query.

```
CREATE VIEW v_sales AS
SELECT city, SUM(amount) AS total
FROM orders GROUP BY city;

CREATE MATERIALIZED VIEW mv_sales AS
SELECT city, SUM(amount) AS total
FROM orders GROUP BY city;

REFRESH MATERIALIZED VIEW mv_sales;
```

Technique (Step-by-Step): 1. Use a view to reuse a query or to hide complex joins. 2. Use a materialized view when the query is slow and data can be a little old. 3. Set a schedule to refresh the materialized view. 4. MySQL has no native materialized views.

Interview Answer: “A view is a saved query, always up to date. A materialized view stores the result, so it is faster but needs refreshing. I use materialized views for heavy reports.”

Follow-up Qs:
- **Q:** Does a view store data? **A:** No. Only a materialized view stores data.
- **Q:** Why use a view? **A:** To simplify queries and to hide columns for security.

Common Mistake: Forgetting to refresh a materialized view and reporting old numbers.

Related: Related to CTEs.

Stored Procedure, Function and Trigger

Definition: A stored procedure is saved SQL code you can run by name. A function is saved code that returns a value. A trigger runs automatically when an event happens, like an INSERT.

Simple Explanation: Procedures do tasks (like a nightly cleanup). Functions are used inside queries. Triggers are used for audit logs and automatic rules.

Example: A procedure that raises salaries by a percent, and a trigger that logs deletes.

```
-- MySQL style procedure
CREATE PROCEDURE raise_salary(IN pct DECIMAL(5,2))
BEGIN
  UPDATE employees SET salary = salary * (1 + pct/100);
END;

CALL raise_salary(10);
```

Technique (Step-by-Step): 1. Use a procedure for a task you repeat. 2. Use a function when you need a value inside SELECT. 3. Use a trigger for automatic actions, and keep them simple. 4. Syntax differs for each database.

Interview Answer: “A stored procedure is reusable SQL I call by name. A function returns a value and can be used in a query. A trigger fires automatically on an event, which is useful for audit logs.”

Follow-up Qs:
- **Q:** Procedure vs function? **A:** A function must return a value and can be used in SELECT. A procedure is called with CALL or EXEC and may return nothing.
- **Q:** Why careful with triggers? **A:** They run hidden, so they can surprise people and slow down writes.

Common Mistake: Putting too much hidden logic in triggers.

Related: Part of advanced SQL.

Query Optimization Tips

Definition: Query optimization means writing queries so they run faster and use fewer resources.

Simple Explanation: Most slow queries come from reading too much data or from blocking the use of indexes.

Example: Rewrite a slow query.

```
-- Slow: function on column blocks the index
WHERE YEAR(order_date) = 2024

-- Fast: plain range
WHERE order_date >= '2024-01-01'
  AND order_date <  '2025-01-01'
```

Technique (Step-by-Step): 1. Select only the columns you need, not *. 2. Filter early with WHERE. 3. Avoid functions on indexed columns in WHERE. 4. Index the join and filter columns. 5. Use UNION ALL instead of UNION when possible. 6. Check EXPLAIN.

Interview Answer: “To speed up a query I select only needed columns, filter early, avoid functions on indexed columns, add indexes on join keys and check the execution plan.”

Follow-up Qs:
- **Q:** First thing you do on a slow query? **A:** Run EXPLAIN to see the plan.
- **Q:** Why avoid SELECT *? **A:** It reads extra data and can stop index-only reads.

Common Mistake: Adding indexes everywhere without checking the plan.

Related: Combines indexes and EXPLAIN.

Level 10 - Quick Revision

Topic	Key point
Clustered index	Sets row order; one per table
Non-clustered index	Separate lookup; many per table
EXPLAIN	Shows the query plan
View	Saved query; always fresh
Materialized view	Stored result; needs refresh
Procedure / Function / Trigger	Saved task / returns value / auto-runs on event

Key rule: Do not wrap an indexed column in a function inside WHERE.

Top Interview Questions: (1) Clustered vs non-clustered index. (2) How do you find why a query is slow? (3) View vs materialized view.

Practical Scenarios: (1) A report takes 5 minutes - EXPLAIN, then index. (2) Hide salary column from analysts - View. (3) Keep an audit of deleted rows - Trigger.

LEVEL 11 - CLASSIC INTERVIEW PROBLEMS

Nth Highest Salary

Definition: Find the Nth highest distinct salary from the employees table.

Simple Explanation: This is the most asked SQL question. The safe way is DENSE_RANK, because it handles ties. You can also use a simple subquery for the 2nd highest.

Example: Find the 3rd highest salary.

```
-- Method 1: DENSE_RANK (best)
SELECT DISTINCT salary FROM (
  SELECT salary,
         DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees) t
WHERE rnk = 3;

-- Method 2: 2nd highest without window
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Technique (Step-by-Step): 1. Rank salaries high to low with DENSE_RANK. 2. Put that in a subquery or CTE. 3. Filter rnk = N. 4. Use DISTINCT in case of ties. 5. Mention the LIMIT 1 OFFSET N-1 method too.

Interview Answer: “I use DENSE_RANK to rank the salaries and then filter for N. It handles ties properly. For the second highest, I can also use MAX with a subquery.”

Follow-up Qs:
- **Q:** Why DENSE_RANK and not RANK? **A:** RANK skips numbers after ties, so the Nth value could be missed.
- **Q:** Other method? **A:** SELECT DISTINCT salary ... ORDER BY salary DESC LIMIT 1 OFFSET N-1.

Common Mistake: Using ROW_NUMBER or RANK, which can give wrong results when salaries tie.

Related: Uses DENSE_RANK (Level 5).

Find and Delete Duplicate Rows

Definition: Find rows that repeat in a column, and remove the extra copies.

Simple Explanation: To find duplicates, use GROUP BY with HAVING COUNT > 1. To delete and keep one, use ROW_NUMBER and delete rows where the number is more than 1.

Example: Find duplicate emails in the users table, then remove extras.

```
-- Find
SELECT email, COUNT(*) AS cnt
FROM users
GROUP BY email
HAVING COUNT(*) > 1;

-- Delete extras, keep the smallest id (PostgreSQL)
WITH d AS (
  SELECT id, ROW_NUMBER() OVER (
         PARTITION BY email ORDER BY id) AS rn
  FROM users)
DELETE FROM users
WHERE id IN (SELECT id FROM d WHERE rn > 1);
```

Technique (Step-by-Step): 1. Find duplicates with GROUP BY and HAVING COUNT(*) > 1. 2. Number each group with ROW_NUMBER, partitioned by the duplicate column. 3. Delete rows where the number is greater than 1. 4. Run a SELECT first to check, and use a transaction.

Interview Answer: “I find duplicates with GROUP BY and HAVING COUNT greater than 1. To delete them, I number rows with ROW_NUMBER per group and remove everything numbered above 1, after testing with a SELECT.”

Follow-up Qs:
- **Q:** How to keep the newest row instead? **A:** Use ORDER BY id DESC inside ROW_NUMBER.
- **Q:** Safe way to try it? **A:** Run a SELECT first, and do the delete inside a transaction.

Common Mistake: Deleting before checking which rows will be removed.

Related: Uses ROW_NUMBER and GROUP BY.

Employees Earning More Than Their Manager

Definition: Find employees whose salary is higher than their own manager's salary.

Simple Explanation: This uses a self join. One copy of the table is the employee, the other copy is the manager.

Example: Show such employees.

```
SELECT e.name AS employee, e.salary,
       m.name AS manager,  m.salary AS mgr_salary
FROM employees e
JOIN employees m
  ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Technique (Step-by-Step): 1. Join the table to itself with two aliases. 2. Join e.manager_id to m.emp_id. 3. In WHERE, compare e.salary to m.salary.

Interview Answer: “I do a self join, where one copy is the employee and the other is their manager. Then I filter where the employee's salary is greater than the manager's.”

Follow-up Qs:
- **Q:** Why INNER JOIN here? **A:** Employees without a manager cannot be compared, so we drop them.
- **Q:** Other way? **A:** A correlated subquery on manager_id.

Common Mistake: Joining on the wrong column, like emp_id to emp_id.

Related: Uses SELF JOIN (Level 3).

Top N per Group (Top 3 Salaries per Department)

Definition: Find the top N rows inside each group, like the 3 highest paid employees in every department.

Simple Explanation: Use a window function with PARTITION BY, and then filter on the rank in an outer query.

Example: Top 3 salaries in each department.

```
SELECT dept, name, salary FROM (
  SELECT dept, name, salary,
         DENSE_RANK() OVER (PARTITION BY dept
                            ORDER BY salary DESC) AS rnk
  FROM employees) t
WHERE rnk <= 3;
```

Technique (Step-by-Step): 1. Rank within each department using PARTITION BY dept. 2. Order by salary DESC. 3. Wrap in a subquery or CTE. 4. Filter rnk <= 3. 5. Use ROW_NUMBER if you want exactly 3 rows, or DENSE_RANK if ties should all show.

Interview Answer: “I rank rows inside each department with PARTITION BY and DENSE_RANK, then filter for rank 3 or lower in an outer query.”

Follow-up Qs:
- **Q:** Department-wise highest salary? **A:** Use the same query with rnk = 1.
- **Q:** ROW_NUMBER or DENSE_RANK here? **A:** ROW_NUMBER for exactly N rows. DENSE_RANK to include ties.

Common Mistake: Trying to filter the rank in the same query's WHERE.

Related: Uses window functions (Level 5).

Customers Who Never Ordered

Definition: Find rows in one table that have no matching rows in another table.

Simple Explanation: This is the anti-join pattern. Use LEFT JOIN with IS NULL, or NOT EXISTS.

Example: Customers with zero orders.

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN orders o
  ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;

-- or
SELECT * FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.customer_id);
```

Technique (Step-by-Step): 1. LEFT JOIN the customers to orders. 2. Filter where an orders column that cannot be NULL is NULL. 3. Or use NOT EXISTS. 4. Avoid NOT IN if NULLs are possible.

Interview Answer: “I use a LEFT JOIN and keep the rows where the order id is NULL. That gives customers with no orders. NOT EXISTS gives the same result.”

Follow-up Qs:
- **Q:** Why not NOT IN? **A:** If the subquery returns a NULL, NOT IN returns nothing.
- **Q:** Which is fastest? **A:** Often they perform the same. Check EXPLAIN.

Common Mistake: Putting the IS NULL check on a column that can be NULL anyway.

Related: Uses anti-join (Level 3).

Consecutive Days (Gaps and Islands)

Definition: Find streaks of consecutive dates, like users who logged in several days in a row.

Simple Explanation: The trick: subtract the row number from the date. For consecutive dates, the result is the same for the whole streak. Group by that result.

Example: Find each user's login streaks (PostgreSQL syntax).

```
SELECT user_id,
       MIN(login_date) AS start_date,
       MAX(login_date) AS end_date,
       COUNT(*)        AS streak_days
FROM (
  SELECT user_id, login_date,
         login_date - (ROW_NUMBER() OVER (
             PARTITION BY user_id ORDER BY login_date)
             * INTERVAL '1 day') AS grp
  FROM (SELECT DISTINCT user_id, login_date FROM logins) x
) t
GROUP BY user_id, grp
HAVING COUNT(*) >= 3;
```

Technique (Step-by-Step): 1. Remove duplicate dates per user. 2. Number the rows per user in date order with ROW_NUMBER. 3. Subtract that number (as days) from the date to get a group key. 4. GROUP BY the user and group key. 5. Count rows for the streak length.

Interview Answer: “For streaks, I number the dates with ROW_NUMBER and subtract the number from the date. Dates in one streak give the same result, so I group by it to find start, end and length.”

Follow-up Qs:
- **Q:** Why DISTINCT first? **A:** Two logins on the same date would break the numbering.
- **Q:** Another method? **A:** Use LAG to compare each date with the previous date and mark where a new streak starts.

Common Mistake: Forgetting to remove duplicate dates before numbering.

Related: Uses ROW_NUMBER and GROUP BY.

Month-over-Month Growth

Definition: Calculate how much revenue changed from one month to the next, in percent.

Simple Explanation: First get monthly totals in a CTE. Then use LAG to bring in last month's value and calculate the change.

Example: Monthly revenue and growth percent.

```
WITH m AS (
  SELECT DATE_TRUNC('month', order_date) AS month,
         SUM(amount) AS revenue
  FROM orders GROUP BY 1)
SELECT month, revenue,
       LAG(revenue) OVER (ORDER BY month) AS prev_rev,
       ROUND(100.0 * (revenue - LAG(revenue) OVER (ORDER BY month))
             / NULLIF(LAG(revenue) OVER (ORDER BY month), 0), 2)
         AS growth_pct
FROM m;
```

Technique (Step-by-Step): 1. Build monthly totals in a CTE with DATE_TRUNC and SUM. 2. Add LAG(revenue) ordered by month. 3. Growth percent = (this month - last month) divided by last month times 100. 4. Use NULLIF to avoid dividing by zero.

Interview Answer: “I first calculate monthly revenue in a CTE, then use LAG to get last month's revenue, and compute the percent change. I use NULLIF to be safe against zero.”

Follow-up Qs:
- **Q:** Why is the first month NULL? **A:** It has no previous month.
- **Q:** How to show year-over-year? **A:** Use LAG(revenue, 12) on monthly data.

Common Mistake: Grouping by month number only, which mixes years.

Related: Uses LAG (Level 5) and DATE_TRUNC (Level 7).

Level 11 - Quick Revision

Problem	Best tool
Nth highest salary	DENSE_RANK
Find duplicates	GROUP BY + HAVING COUNT > 1
Delete duplicates	ROW_NUMBER + DELETE
Earning more than manager	SELF JOIN
Top N per group	DENSE_RANK + PARTITION BY
Never ordered	LEFT JOIN + IS NULL
Consecutive days	Date minus ROW_NUMBER
MoM growth	CTE + LAG

Key rule: Most interview problems are a mix of JOIN, GROUP BY and window functions.

Top Interview Questions: (1) Nth highest salary. (2) Find and delete duplicates. (3) Top 3 per department. (4) Customers with no orders.

Practical Scenarios: (1) Find users with 3 or more login days in a row - gaps and islands. (2) Show month-over-month sales change - LAG. (3) Clean a table that has repeated rows - ROW_NUMBER.

MASTER REVISION - ALL 11 LEVELS AT A GLANCE

Level	Topic	Core Items
1	SQL Basics	SELECT, DISTINCT, WHERE, ORDER BY, LIMIT/TOP, LIKE, IN, BETWEEN, IS NULL
2	Aggregates and Grouping	COUNT, SUM, AVG, MIN/MAX, GROUP BY, HAVING, WHERE vs HAVING, Order of Execution
3	Joins	INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF, Anti-join
4	Subqueries and CTEs	Subquery, Correlated Subquery, EXISTS vs IN, CTE, Recursive CTE
5	Window Functions	PARTITION BY, ROW_NUMBER, RANK, DENSE_RANK, LAG/LEAD, Running Total, NTILE
6	Set Operations and Logic	UNION, UNION ALL, INTERSECT, EXCEPT, CASE WHEN, COALESCE, NULLIF
7	String and Date Functions	SUBSTRING, CONCAT, UPPER/LOWER/TRIM, REPLACE, LENGTH, DATEDIFF, DATEADD, EXTRACT, DATE_TRUNC, CURRENT_DATE
8	DDL, DML and Constraints	CREATE, ALTER, INSERT, UPDATE, DELETE vs TRUNCATE vs DROP, Constraints
9	Design and Transactions	Keys, Normalization, Denormalization, ACID, COMMIT/ROLLBACK
10	Performance and Objects	Indexes, EXPLAIN, Views, Materialized Views, Procedures, Triggers, Optimization
11	Classic Problems	Nth highest salary, duplicates, manager salary, top N per group, never ordered, gaps and islands, MoM growth

INTERVIEW IMPORTANT - DO NOT MISS THESE

- WHERE filters rows before grouping. HAVING filters groups after grouping.
- Run order: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY, LIMIT.
- COUNT(*) counts all rows. COUNT(column) skips NULLs.
- Use IS NULL, never = NULL.
- INNER keeps matches only. LEFT keeps all left rows. FULL OUTER keeps everything.
- Customers who never ordered: LEFT JOIN + IS NULL, or NOT EXISTS.
- Prefer NOT EXISTS over NOT IN when NULLs are possible.
- ROW_NUMBER (unique) vs RANK (gaps after ties) vs DENSE_RANK (no gaps).
- Nth highest salary: use DENSE_RANK.
- UNION removes duplicates. UNION ALL keeps them and is faster.
- DELETE (rows, WHERE, can roll back) vs TRUNCATE (empty table fast) vs DROP (remove table).
- PRIMARY KEY is unique and not NULL. FOREIGN KEY links tables.
- ACID: Atomicity, Consistency, Isolation, Durability.
- Clustered index sets row order (one per table). Non-clustered is separate (many).
- Do not use a function on an indexed column in WHERE.
- Use NULLIF(divisor, 0) to avoid divide-by-zero.

HOW TO USE THIS GUIDE

1. Go level by level in order. Later levels use ideas from earlier levels (for example, Level 11 needs joins and window functions).

2. For each topic, read the Definition and Simple Explanation first. Then type the example query yourself in a SQL tool. This builds real skill.

3. Say the Interview Answer out loud in your own words. Do not just read it.

4. Try the Follow-up Questions before reading the answers.

5. Practice on LeetCode (Database), HackerRank, StrataScratch or DataLemur, mainly joins, GROUP BY and window functions.

6. Before an interview, read only the Quick Revision tables and the Interview Important list.

TOP 100 SQL INTERVIEW QUESTIONS WITH ANSWERS

Read each answer, then say it in your own words. Short code is shown where the interviewer may ask you to write a query.

A. SQL BASICS

Q1. What is SQL?

A: SQL (Structured Query Language) is the language used to store, read, change and delete data in a relational database.

Q2. What is a database and a table?

A: A database is a place where data is stored. A table is a part of the database that stores data in rows and columns.

Q3. What is the difference between SQL and MySQL?

A: SQL is the language. MySQL is a database software that uses the SQL language. Others are PostgreSQL, SQL Server and Oracle.

Q4. What are the types of SQL commands?

A: DDL (CREATE, ALTER, DROP, TRUNCATE), DML (INSERT, UPDATE, DELETE), DQL (SELECT), DCL (GRANT, REVOKE) and TCL (COMMIT, ROLLBACK, SAVEPOINT).

Q5. What is the difference between WHERE and HAVING?

A: WHERE filters rows before grouping and cannot use aggregates. HAVING filters groups after GROUP BY and can use aggregates like COUNT and SUM.

Q6. What is the order of execution of a SQL query?

A: FROM, WHERE, GROUP BY, HAVING, SELECT, DISTINCT, ORDER BY, LIMIT. This is why a SELECT alias works in ORDER BY but not in WHERE.

Q7. What does DISTINCT do?

A: It removes duplicate rows from the result so each value shows only once.

Q8. How do you get the top 5 rows?

A: Sort with ORDER BY and then limit the rows. Use LIMIT 5 in MySQL/PostgreSQL, TOP 5 in SQL Server, or FETCH FIRST 5 ROWS ONLY in Oracle.

Q9. What is the difference between = and LIKE?

A: = matches the exact value. LIKE matches a pattern using % (any characters) and _ (one character).

Q10. How do you check for NULL values?

A: Use IS NULL or IS NOT NULL. column = NULL never works, because NULL means unknown.

Q11. What is NULL? Is it the same as 0 or empty text?

A: No. NULL means the value is missing or unknown. 0 is a number and an empty string is a text with no characters.

Q12. What is an alias?

A: An alias is a temporary name for a column or table, written with AS. It makes results and queries easier to read.

Q13. What is the difference between IN and BETWEEN?

A: IN checks a value against a list of values. BETWEEN checks a value against a range and includes both ends.

Q14. What is the difference between CHAR and VARCHAR?

A: CHAR has a fixed length and pads extra spaces. VARCHAR has a variable length and uses only the space it needs.

Q15. What are SQL data types you use most?

A: INT, DECIMAL, VARCHAR, DATE, TIMESTAMP and BOOLEAN. Pick the smallest type that fits the data.

B. JOINS

Q16. What is a JOIN?

A: A JOIN combines rows from two or more tables using a related column, like customer_id.

Q17. What are the types of joins?

A: INNER, LEFT, RIGHT, FULL OUTER, CROSS and SELF join.

Q18. What is the difference between INNER JOIN and LEFT JOIN?

A: INNER JOIN returns only matching rows. LEFT JOIN returns all rows from the left table and NULL for the right side where there is no match.

Q19. What is a FULL OUTER JOIN?

A: It returns all rows from both tables. Where there is no match, the other side shows NULL. MySQL does not support it directly; use LEFT JOIN UNION RIGHT JOIN.

Q20. What is a CROSS JOIN?

A: It pairs every row of table A with every row of table B. 3 rows and 4 rows give 12 rows. It has no ON condition.

Q21. What is a SELF JOIN?

A: A table joined to itself using two aliases. It is used for hierarchy data like employee and manager.

Q22. How do you find customers who never placed an order?

A: Use a LEFT JOIN and keep rows where the order key is NULL.

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Q23. Why can a LEFT JOIN turn into an INNER JOIN?

A: If you filter the right table in WHERE, rows with NULL are removed. Put that filter in the ON clause instead.

Q24. What happens if you forget the ON condition?

A: You may get a CROSS JOIN (every row with every row), which creates a huge wrong result.

Q25. What is the difference between JOIN and UNION?

A: JOIN adds columns side by side from related tables. UNION stacks rows from two queries on top of each other.

Q26. Can you join more than two tables?

A: Yes. Write one JOIN after another, each with its own ON condition.

Q27. What is an equi join and a non-equi join?

A: An equi join uses = in the ON condition. A non-equi join uses other operators like >, < or BETWEEN.

Q28. What is a natural join?

A: A join that automatically uses all columns with the same name in both tables. It is risky, so most people write ON clearly.

Q29. How do you join on multiple columns?

A: Use AND in the ON clause.

```
SELECT * FROM a
JOIN b ON a.id = b.id AND a.year = b.year;
```

Q30. How do you compare two tables to find rows in one but not the other?

A: Use FULL OUTER JOIN and check for NULLs, or use EXCEPT / NOT EXISTS.

C. AGGREGATES, GROUP BY AND SUBQUERIES

Q31. What are aggregate functions?

A: They calculate one value from many rows: COUNT, SUM, AVG, MIN and MAX.

Q32. What is the difference between COUNT(*), COUNT(column) and COUNT(DISTINCT column)?

A: COUNT(*) counts all rows. COUNT(column) skips NULLs. COUNT(DISTINCT column) counts unique non-NULL values.

Q33. Do aggregate functions ignore NULL?

A: Yes, except COUNT(*). SUM, AVG, MIN, MAX and COUNT(column) skip NULLs.

Q34. What is GROUP BY?

A: It groups rows with the same value so you can calculate one result per group, like total sales per city.

Q35. Can you use a column in SELECT that is not in GROUP BY?

A: Not unless it is inside an aggregate function. Most databases give an error.

Q36. How do you find customers with more than 5 orders?

A: Group by customer and filter with HAVING.

```
SELECT customer_id, COUNT(*) AS cnt
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 5;
```

Q37. What is a subquery?

A: A query written inside another query. The inner query runs first and the outer query uses its result.

Q38. What is a correlated subquery?

A: A subquery that uses a column from the outer query, so it runs once for each outer row. It can be slow on large data.

Q39. What is the difference between EXISTS and IN?

A: IN compares a value to a list. EXISTS checks if any matching row exists and stops at the first match. NOT EXISTS is safer than NOT IN when NULLs are possible.

Q40. Why is NOT IN dangerous?

A: If the list contains a NULL, NOT IN returns no rows at all. Use NOT EXISTS or filter out NULLs.

Q41. What is a CTE?

A: A Common Table Expression is a named temporary result created with WITH. It makes complex queries easy to read.

Q42. CTE vs subquery vs temp table?

A: A CTE and subquery last for one query. A CTE is more readable. A temp table is stored for the session and can be indexed and reused many times.

Q43. What is a recursive CTE?

A: A CTE that calls itself to walk through levels of data, like an employee to manager chain. It has an anchor query and a recursive query joined by UNION ALL.

Q44. How do you find employees earning more than the average salary?

A: Compare salary with a subquery that calculates the average.

```
SELECT name, salary FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q45. How do you find the highest salary in each department?

A: Group by department and use MAX, or use a window function to also show the name.

```
SELECT dept, MAX(salary) FROM employees
GROUP BY dept;
```

D. WINDOW FUNCTIONS

Q46. What is a window function?

A: A function that calculates across related rows but keeps every row in the result, unlike GROUP BY which collapses rows.

Q47. What is the difference between GROUP BY and PARTITION BY?

A: GROUP BY returns one row per group. PARTITION BY keeps all rows and adds the group calculation to each row.

Q48. What is the difference between ROW_NUMBER, RANK and DENSE_RANK?

A: ROW_NUMBER is always unique (1,2,3,4). RANK gives ties the same number and skips after (1,2,2,4). DENSE_RANK gives ties the same number without gaps (1,2,2,3).

Q49. How do you find the Nth highest salary?

A: Use DENSE_RANK in a subquery and filter for N.

```
SELECT DISTINCT salary FROM (
  SELECT salary,
   DENSE_RANK() OVER (ORDER BY salary DESC) rnk
  FROM employees) t
WHERE rnk = 3;
```

Q50. How do you get the top 3 salaries in each department?

A: Use DENSE_RANK with PARTITION BY dept, then filter rnk <= 3.

```
SELECT * FROM (
  SELECT dept, name, salary,
   DENSE_RANK() OVER (PARTITION BY dept
                      ORDER BY salary DESC) rnk
  FROM employees) t
WHERE rnk <= 3;
```

Q51. What do LAG and LEAD do?

A: LAG gets the value from the previous row. LEAD gets the value from the next row. They are used for change-over-time reports.

Q52. How do you calculate a running total?

A: Use SUM with OVER and ORDER BY.

```
SELECT sale_date, amount,
  SUM(amount) OVER (ORDER BY sale_date) AS running_total
FROM daily_sales;
```

Q53. How do you calculate month-over-month growth?

A: Get monthly totals, use LAG for the previous month, then calculate (this - last) / last * 100. Use NULLIF to avoid dividing by zero.

Q54. Can you use a window function in WHERE?

A: No. Put it in a CTE or subquery first and filter in the outer query.

Q55. What does NTILE do?

A: It splits ordered rows into N equal groups, like quartiles with NTILE(4).

E. FUNCTIONS, NULL HANDLING AND SET OPERATIONS

Q56. What is the difference between UNION and UNION ALL?

A: UNION removes duplicates and is slower. UNION ALL keeps duplicates and is faster.

Q57. What are INTERSECT and EXCEPT?

A: INTERSECT returns rows found in both queries. EXCEPT (MINUS in Oracle) returns rows in the first query that are not in the second.

Q58. What rules apply to UNION?

A: Both queries need the same number of columns, in the same order, with compatible data types.

Q59. What is CASE WHEN?

A: It is if-then-else in SQL. It checks conditions from top to bottom and returns the result of the first true one.

```
SELECT name,
 CASE WHEN salary >= 100000 THEN 'High'
      WHEN salary >= 50000  THEN 'Medium'
      ELSE 'Low' END AS band
FROM employees;
```

Q60. What is COALESCE?

A: It returns the first non-NULL value from a list. It is used to replace NULL with a default value.

Q61. What is NULLIF and why is it useful?

A: NULLIF returns NULL if two values are equal. It is used to avoid divide-by-zero: x / NULLIF(y, 0).

Q62. COALESCE vs IFNULL?

A: IFNULL takes two values and is MySQL-specific. COALESCE takes many values and works in all databases.

Q63. How do you count only some rows using CASE?

A: Put CASE inside SUM or COUNT.

```
SELECT SUM(CASE WHEN status = 'Paid' THEN 1 ELSE 0 END)
       AS paid_orders
FROM orders;
```

Q64. How do you get only the year or month from a date?

A: Use EXTRACT(YEAR FROM date) or YEAR(date). For grouping by month, use DATE_TRUNC('month', date) in PostgreSQL.

Q65. How do you find the days between two dates?

A: Use DATEDIFF in MySQL and SQL Server, or subtract the dates in PostgreSQL.

Q66. How do you get the last 30 days of data?

A: Compare the date to today minus 30 days.

```
WHERE order_date >= CURRENT_DATE - INTERVAL '30 days'
```

Q67. How do you remove extra spaces from text?

A: Use TRIM. Also use UPPER or LOWER to make the case the same before comparing or joining.

Q68. How do you find the length of a text?

A: Use LENGTH (MySQL, PostgreSQL, Oracle) or LEN (SQL Server). Spaces are counted.

Q69. How do you join first name and last name?

A: Use CONCAT(first_name, ' ', last_name), or || in PostgreSQL and Oracle.

Q70. How do you cut part of a text?

A: Use SUBSTRING(text, start, length), LEFT or RIGHT. Positions start at 1.

F. DDL, DML, KEYS AND DESIGN

Q71. What is the difference between DELETE, TRUNCATE and DROP?

A: DELETE removes chosen rows with WHERE and can be rolled back. TRUNCATE empties the whole table fast but keeps its structure. DROP removes the table completely.

Q72. What is the difference between DDL and DML?

A: DDL changes the structure (CREATE, ALTER, DROP). DML changes the data (INSERT, UPDATE, DELETE).

Q73. What is a primary key?

A: A column (or columns) that uniquely identifies each row. It cannot be NULL and a table has only one.

Q74. What is a foreign key?

A: A column that points to the primary key of another table. It makes sure the value exists in the parent table.

Q75. PRIMARY KEY vs UNIQUE KEY?

A: A table has one primary key and it cannot be NULL. A table can have many unique keys and they can allow a NULL.

Q76. What is a composite key?

A: A key made of two or more columns together, like order_id and product_id.

Q77. What are constraints?

A: Rules that protect data: NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK and DEFAULT.

Q78. What is normalization?

A: Organizing data into related tables to remove repeated data. 1NF: one value per cell. 2NF: no partial dependency. 3NF: no dependency between normal columns.

Q79. What is denormalization and when is it used?

A: Adding repeated data on purpose to make reads faster. It is used in reporting and data warehouses.

Q80. What are ACID properties?

A: Atomicity (all or nothing), Consistency (rules stay true), Isolation (transactions do not disturb each other), Durability (committed data stays safe).

Q81. What is a transaction? What are COMMIT and ROLLBACK?

A: A transaction is a group of steps treated as one unit. COMMIT saves the changes. ROLLBACK cancels them.

Q82. What happens if you run UPDATE without WHERE?

A: All rows in the table are updated. Always test with SELECT first and use a transaction.

Q83. What is the difference between a view and a table?

A: A table stores data. A view is a saved query that works like a virtual table and does not store data.

Q84. What is a materialized view?

A: A view whose result is stored on disk. It reads faster but needs a refresh, so it can be old.

Q85. What is a stored procedure and a function?

A: A stored procedure is saved SQL you run by name. A function returns a value and can be used inside a query.

Q86. What is a trigger?

A: Code that runs automatically when an event happens on a table, like INSERT or DELETE. It is often used for audit logs.

G. PERFORMANCE

Q87. What is an index?

A: A lookup structure that helps the database find rows fast without reading the whole table, like the index of a book.

Q88. Clustered vs non-clustered index?

A: A clustered index decides the physical row order and a table has only one. A non-clustered index is a separate structure and a table can have many.

Q89. What are the downsides of indexes?

A: They use extra space and slow down INSERT, UPDATE and DELETE.

Q90. How do you find why a query is slow?

A: Run EXPLAIN to see the plan. Look for full table scans, then add an index or rewrite the query.

Q91. Why can an index stop working?

A: Using a function on the indexed column in WHERE, like WHERE YEAR(date) = 2024, or a leading wildcard like LIKE '%abc'.

Q92. How do you optimize a slow query?

A: Select only needed columns, filter early, index join and filter columns, avoid functions on indexed columns, use UNION ALL instead of UNION when possible, and check EXPLAIN.

H. CLASSIC CODING QUESTIONS

Q93. How do you find duplicate rows?

A: Group by the column and keep groups with a count above 1.

```
SELECT email, COUNT(*)
FROM users
GROUP BY email
HAVING COUNT(*) > 1;
```

Q94. How do you delete duplicate rows and keep one?

A: Number the rows per group with ROW_NUMBER and delete those numbered above 1.

```
WITH d AS (
 SELECT id, ROW_NUMBER() OVER (
   PARTITION BY email ORDER BY id) rn
 FROM users)
DELETE FROM users
WHERE id IN (SELECT id FROM d WHERE rn > 1);
```

Q95. How do you find employees earning more than their manager?

A: Use a self join and compare salaries.

```
SELECT e.name
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Q96. How do you find the second highest salary without a window function?

A: Use MAX with a subquery.

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Q97. How do you find users who logged in on 3 days in a row?

A: Subtract ROW_NUMBER from the date. Dates in one streak give the same value. Group by it and keep groups with count 3 or more. Remove duplicate dates first.

Q98. How do you find customers who bought in 2023 but not in 2024?

A: Use EXCEPT, or a LEFT JOIN with IS NULL, or NOT EXISTS.

```
SELECT customer_id FROM orders_2023
EXCEPT
SELECT customer_id FROM orders_2024;
```

Q99. How do you get the latest order for each customer?

A: Number orders per customer by date descending and keep number 1.

```
SELECT * FROM (
 SELECT *, ROW_NUMBER() OVER (
   PARTITION BY customer_id
   ORDER BY order_date DESC) rn
 FROM orders) t
WHERE rn = 1;
```

Q100. How do you swap values or flip a column using SQL?

A: Use CASE WHEN, for example to change 'M' to 'F' and 'F' to 'M' in one UPDATE.

```
UPDATE people SET gender =
 CASE WHEN gender = 'M' THEN 'F' ELSE 'M' END;
```